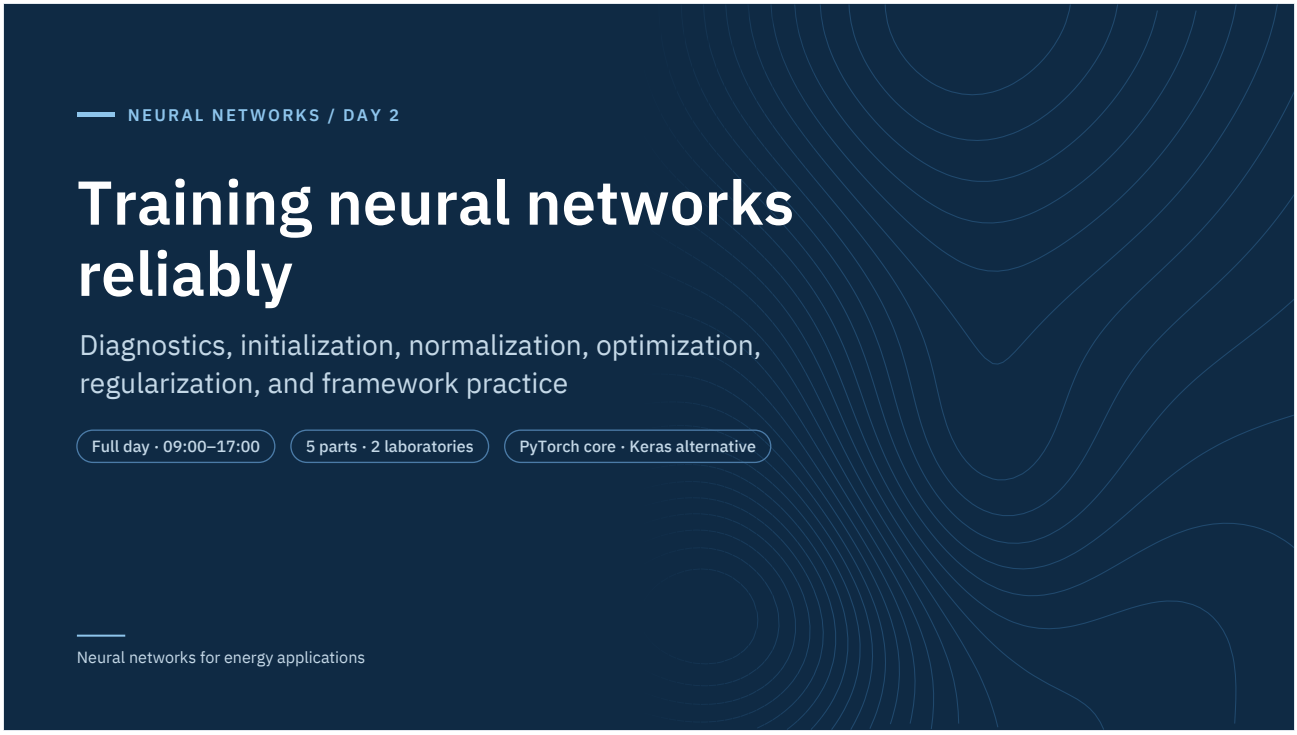
Training neural networks reliably

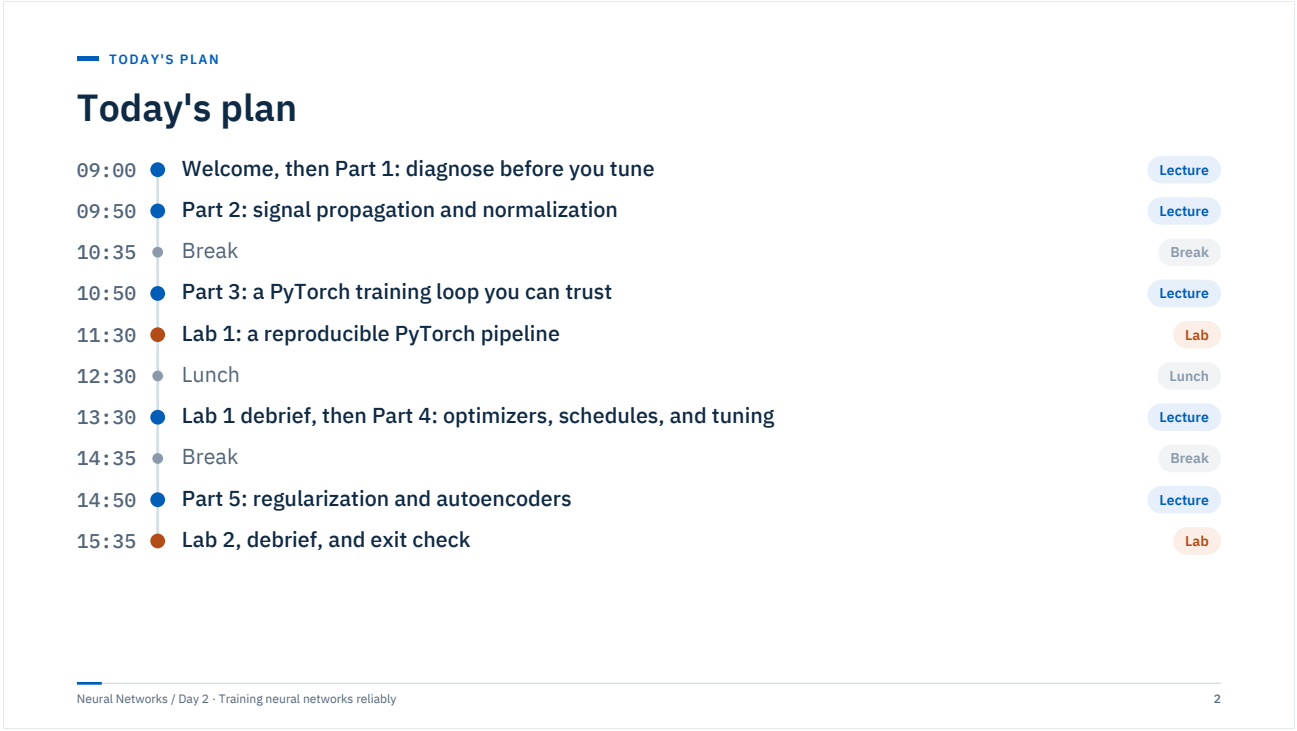


Prerequisite: Day 1, including tensor shapes, logits, losses, the chain rule and the basic training loop. Day 1 showed how a network learns; today is about making that learning reliable, diagnosable and repeatable.

Open by asking the room: "Your training loss went down every epoch. What have you proved?" Collect two or three answers. The point to land: a decreasing training loss is necessary but not sufficient evidence of useful learning. It says nothing about validation performance, nothing about whether the pipeline is wired correctly, and nothing about whether the result survives a different seed.

Explain the shape of the day: five teaching parts, two laboratories, and a fixed diagnostic habit that runs through all of it. PyTorch is the core route; Keras appears as a translation so that learners who meet it at work can map the same contract. [3 min]

Today's plan



Walk through the plan in one minute. The morning builds the diagnostic habit (Part 1), explains why deep networks become hard to train and how initialization, normalization and skip connections help (Part 2), then turns everything into a PyTorch loop you can trust (Part 3) before the first laboratory at 11:30.

After lunch, start with a ten-minute debrief of Lab 1, then optimization and tuning (Part 4), a break, and regularization plus a short introduction to autoencoders (Part 5). Lab 2 (15:35–16:30) offers two tracks: a regularization comparison on MNIST or an autoencoder for anomaly scoring. From 16:30 the Lab 2 debrief, key takeaways and a written exit check close the day by about 16:55.

Timing guide: the welcome and warm-up take 15 minutes inside the first row; Part 1 runs 09:15–09:50. If Part 2 overruns, shorten the normalization comparison slide rather than the exercise. [1 min]

What you will be able to do

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Run a fixed diagnostic sequence: initial loss, one-batch fit, gradient flow, learning curves.
2. Explain how initialization, normalization, and skip connections keep signals at a trainable scale.
3. Write a PyTorch loop with correct train and eval modes, a copied checkpoint, and a reload check.
4. Choose an optimizer, learning rate, schedule, and batch size from evidence, not habit.
5. Match weight decay, dropout, early stopping, and augmentation to the symptom they treat.
6. Explain how an autoencoder turns reconstruction error into an anomaly score, and its limits.

Neural Networks / Day 2 · Training neural networks reliably 3

Read the outcomes aloud and point out that each one is observable: by the exit check, learners should be able to perform or explain each of them without notes.

Outcome 1 is the habit that ties the day together. Outcomes 2 and 5 are about mechanisms, not recipes: we want learners to know why a technique helps, so they can tell when it will not. Outcome 3 is assessed in Lab 1, where the reload check is part of the submitted evidence. Outcome 4 is assessed in the Part 4 exercises, and outcome 6 in the autoencoder track of Lab 2.

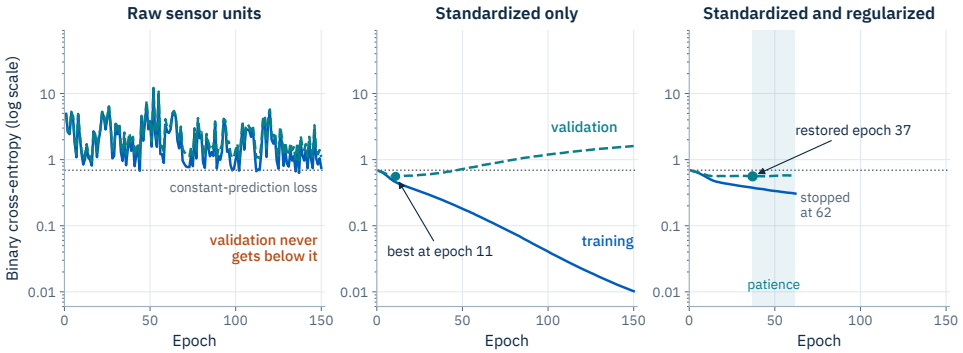
Ask learners to mark the outcome they most need for their own work; revisit these marks at the exit check. [2 min]

Same network, same data, three different outcomes

MOTIVATING CASE

Same network, same data, three different outcomes

A 12-input MLP learns a pump-failure flag from a synthetic ESP table with 240 training rows. Only the training setup changes.



Computed: synthetic ESP-style data, seeds 1 and 2 (raw pressures in kPa next to vibration in g), MLP 12-64-64-1, AdamW, torch seed 0; the dotted line is the loss of always predicting the training base rate.

Neural Networks / Day 2 · Training neural networks reliably 4

This is the motivating case for the day. The data are synthetic but shaped like an electrical submersible pump (ESP) table: intake pressure in kilopascals (around 9,000), motor current in amps, drive frequency, vibration in g (around 0.3) and temperature, plus seven nuisance columns. The architecture and optimizer are identical in all three panels.

Left: the raw units are left in place. The loss jumps between 1 and 10 and the validation loss never gets below 0.69, the loss of simply predicting the base rate (47% events). The network is worse than a constant. Middle: the same data standardized. Training loss falls to 0.01 while validation loss is best at epoch 11 (0.559) and then climbs to 1.6: the final model memorizes. Right: standardized, with weight decay and dropout, and early stopping with patience 25. The best validation loss is the same 0.559, but the curves stay flat and the restored checkpoint is safe.

Message: most of today is about moving from the left and middle panels to the right one, and knowing which evidence tells you where you are. [3 min]

Warm-up: which run would you trust, and why?

PRACTICE

5 min Pairs

Warm-up: which run would you trust, and why?

Look at the three runs on the previous slide.

1. Which run would you hand to an operations team, and what evidence would you ask for first?
2. The middle run reaches a training loss of 0.01. What does that number prove?
3. Name the first setting you would check in the left-hand run.

Neural Networks / Day 2 · Training neural networks reliably 5

Give pairs three minutes, then take one answer per question. [5 min]

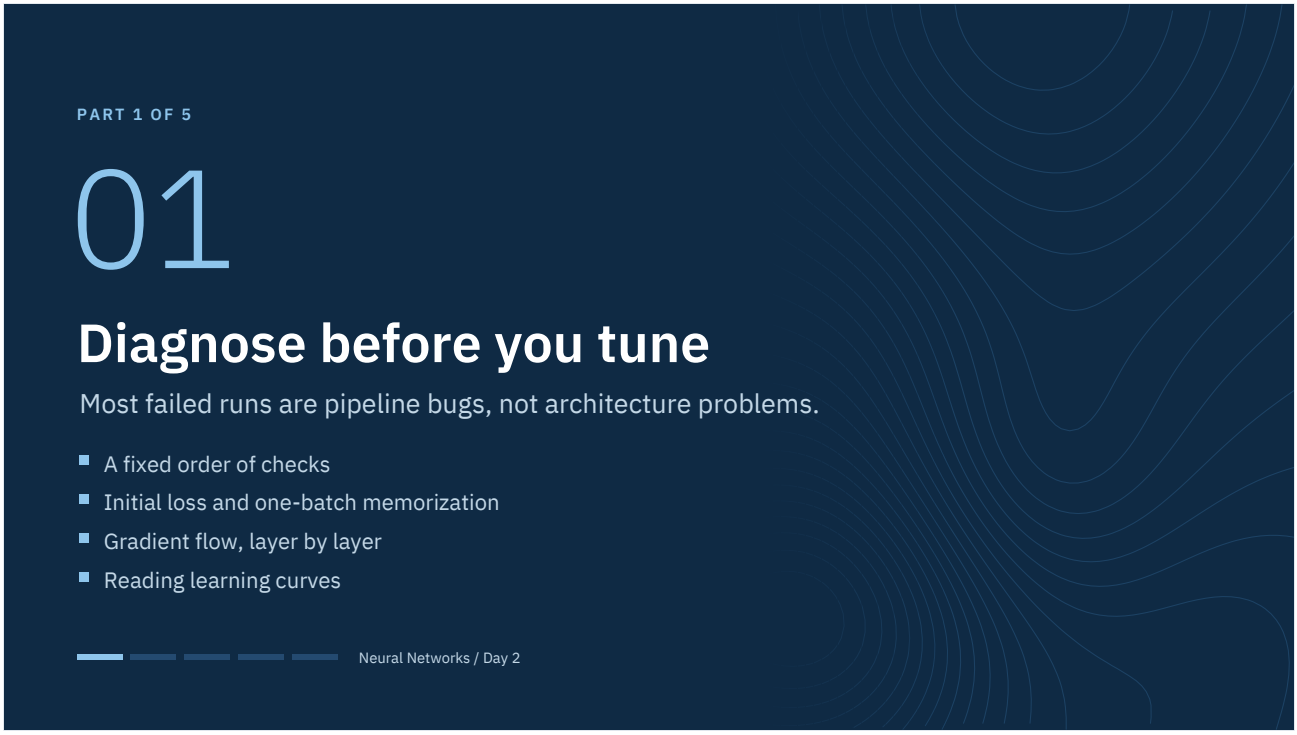
Answer 1: the right-hand run, but only after checking it on data it has not influenced. Its checkpoint was chosen on the validation rows, so the validation score is optimistic; a separate test set or a later time period is still needed. Ask for learning curves, the split design, a baseline (for example logistic regression) and the result over several seeds.

Answer 2: very little. A training loss of 0.01 proves the network can fit those 240 rows, including their noise. It says nothing about new rows; the validation curve shows the model got worse after epoch 11. Memorization is easy for a network with thousands of parameters and a few hundred rows.

Answer 3: input scaling. Pressures near 9,000 kPa and vibration near 0.3 g put the first layer's weighted sums on wildly different scales, so the first updates are dominated by one column and the loss jumps. Standardizing with statistics fitted on the training rows fixes this panel; learning rate is the second suspect.

Common error: blaming the architecture first. None of the three panels needs a different architecture.

Diagnose before you tune



Part 1 runs from about 09:15 to 09:50. The goal is a habit: before changing any hyperparameter, run a short sequence of cheap checks that catch the most common bugs. Each check has an expected result that can be computed in advance, which is what makes it a check rather than a hope.

Frame it with an energy example: a data scientist spends two days trying deeper networks on a well-log facies problem, and the real problem is that the label column was shifted by one row after a join. The checks in this part would have found it in ten minutes. [1 min]

A diagnostic sequence saves time

DIAGNOSE BEFORE YOU TUNE

A diagnostic sequence saves time

Check	Expected evidence	If it fails
Data and labels	Correct shapes, units, and alignment	Repair the input path
One forward pass	Finite outputs and plausible loss	Inspect output-loss contract
Tiny batch fit	Training loss falls substantially	Inspect gradients and optimizer
Validation fit	Generalization beyond fitted rows	Review capacity, split, and regularization

Run the checks in this order; each one assumes the previous one passed.

◆ A failure early in the list makes every later result meaningless.

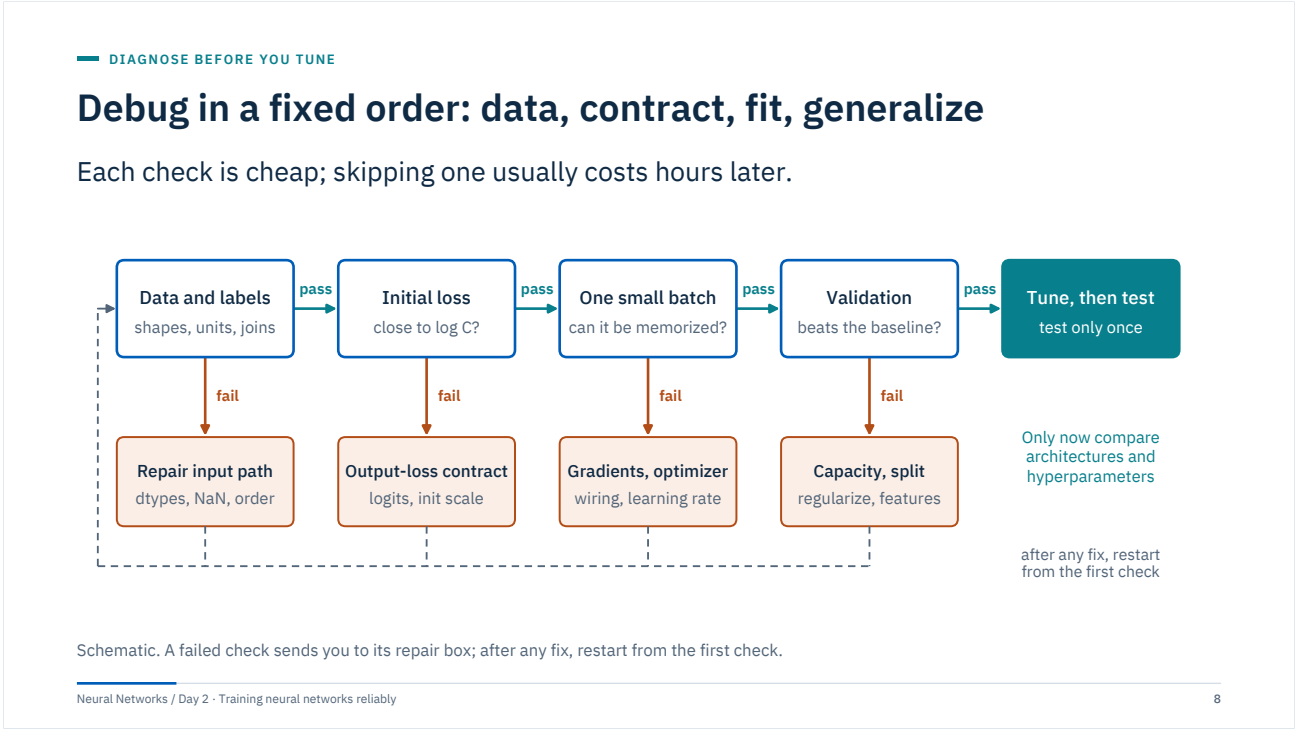
Neural Networks / Day 2 · Training neural networks reliably 7

Walk down the table. Data and labels first: shapes, units, missing values, and whether each target still belongs to the right row after every join and sort. Then one forward pass: outputs must be finite and the loss must be plausible for an untrained model (next slides give the exact number). Then fit a tiny batch: a correctly wired network can drive the loss on a few rows close to zero. Only then does validation performance tell you anything about generalization.

Fitting a tiny batch is a diagnostic, not an assessment. Disable heavy regularization, such as dropout and weight decay, while testing whether the model can fit a small batch, then restore the intended settings for real experiments.

Ask: why this order? Because a failure early in the list makes every later result meaningless: a validation score computed on misaligned labels is noise. [2 min]

Debug in a fixed order: data, contract, fit, generalize



This flowchart is the table as a procedure. The top row is the happy path; each check has one repair box beneath it. Two rules matter. First, after any fix, restart from the first check, because fixes interact: repairing a scaling bug changes the initial loss, and a new initial loss changes what the one-batch test should show. Second, only when all four checks pass does it make sense to compare architectures or tune hyperparameters.

Relate it to the motivating case: the left-hand run fails the initial-loss check (the loss starts far above log 2 because raw units produce huge logits). The middle run passes the first three checks and fails at validation: its repair box is capacity, split and regularization.

Transition: the next two slides make the second and third checks quantitative. [2 min]

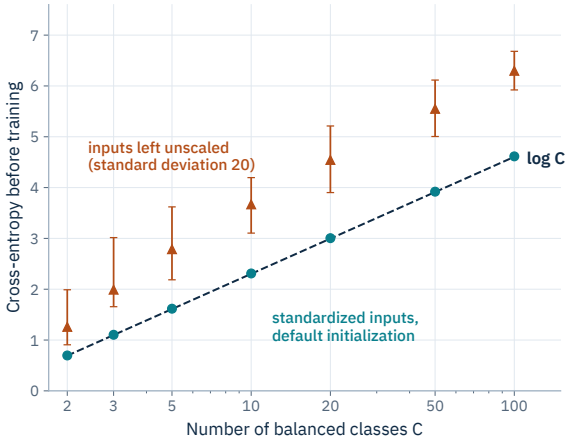
Initial loss should be close to log C

DIAGNOSE BEFORE YOU TUNE

Initial loss should be close to log C

$$L_0 = -\log \frac{1}{C} = \log C$$

- Untrained logits are near zero: predictions near uniform
- Binary: log 2 = 0.693; ten classes: log 10 = 2.303
- Far higher: logits too large, or a wrong loss contract
- Imbalanced classes: use the entropy of the class prior



Computed: 20 seeds per point, PyTorch default initialization; the orange points use the same inputs multiplied by 20.

Neural Networks / Day 2 · Training neural networks reliably 9

Define the symbols: C is the number of classes, L0 the cross-entropy before any training. With near-zero logits, softmax gives each class probability 1/C, so the loss per example is -log(1/C) = log C. For a binary model with one logit and BCEWithLogitsLoss the same argument gives log 2 = 0.693.

The teal points are real untrained networks with standardized inputs: the mean initial loss is 0.70 for two classes, 2.31 for ten, 4.61 for a hundred, all on the log C line. The orange points feed the same networks inputs with standard deviation 20, as happens when a column is left in raw units: the loss is 1.26, 3.67 and 6.30. A value well above log C means the network is confidently wrong before it has learned anything.

For imbalanced classes a well-initialized output bias gives the entropy of the class prior, for example about 0.199 for a 5% event rate; either reference is fine as long as you know which one you expect. [3 min]

Put the sanity checks in code, not in memory

DIAGNOSE BEFORE YOU TUNE

Put the sanity checks in code, not in memory

```python
import math

xb, yb = next(iter(train_loader))
assert xb.shape == (16, 60) and yb.shape == (16, 1)
assert torch.isfinite(xb).all()
assert set(yb.unique().tolist()) <= {0.0, 1.0}

model.eval()
with torch.no_grad():
    loss0 = criterion(model(xb), yb).item()
print(f"initial loss {loss0:.3f}   log 2 = {math.log(2):.3f}")
```

OUTPUT

```
initial loss 0.703   log 2 = 0.693
```

Lab 1 data and untrained model (seed 42); one batch of 16 rows varies by a few hundredths.

Neural Networks / Day 2 · Training neural networks reliably 10

This is the first check turned into code that runs every time, not only when something looks wrong. The asserts encode the prediction contract for the Sonar lab: 60 standardized features, one target column per row, targets that are exactly 0 or 1, no NaN or infinity.

The initial loss is computed in eval mode so dropout does not add noise, and without gradient tracking because we are only measuring. The output comes from the Lab 1 model with torch seed 42 and the first batch of the seeded training loader (generator seed 42, batch size 16, shown in Part 3): 0.703 on these 16 rows, close to log 2 = 0.693; on all 124 training rows the same untrained model gives 0.693. A single small batch will differ from log 2 by a few hundredths, which is fine. A value of 3 or 30 would not be.

Common error: running this check after the first optimizer step, when the loss has already moved. Run it before training. [2 min]

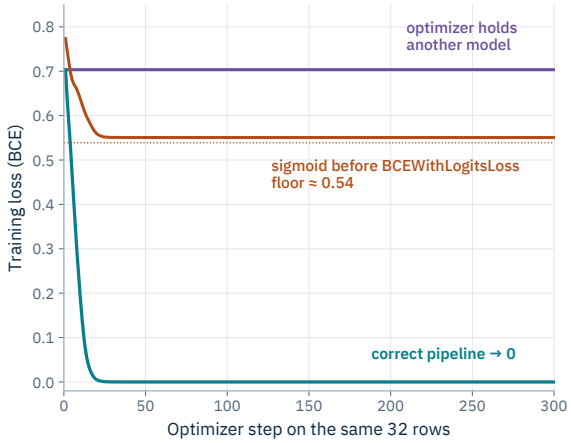
A healthy pipeline can memorize one small batch

DIAGNOSE BEFORE YOU TUNE

A healthy pipeline can memorize one small batch

- Train on the same 32 rows for a few hundred steps
- Switch off dropout and weight decay for this test
- Loss near zero: forward, loss, backward, and step are wired correctly
- A floor or a flat line points to a contract or optimizer bug

◆ Memorizing one batch proves the plumbing, not generalization.



Computed: 32 rows of the synthetic ESP table (seed 1), MLP 12-64-64-1, Adam lr 0.01, torch seed 0; both bugs injected on purpose.

Neural Networks / Day 2 · Training neural networks reliably 11

A network with a few thousand parameters should be able to memorize 32 rows exactly. If it cannot, something between the data and the weights is broken. Teal: the correct pipeline reaches a loss near zero within about 30 steps.

Orange: a Sigmoid layer was left at the end of the model while the loss is BCEWithLogitsLoss, which applies its own sigmoid. The double squashing means the effective probability can never go below 0.5 or above sigmoid(1) = 0.731, so the loss has a floor. For this batch (13 events among 32 rows) the floor is 13/32 × 0.313 + 19/32 × 0.693 = 0.54; the curve stalls at 0.55 because the inner sigmoid saturates. Purple: the optimizer was built from a different model instance, so step() updates tensors that the forward pass never uses; the loss stays at its initial value of about 0.70.

Note that a network can also memorize random labels (Zhang et al., 2017), so this test cannot detect label errors; that is what the data check is for. [3 min] @source https://arxiv.org/abs/1611.03530

Check that every layer receives a gradient

DIAGNOSE BEFORE YOU TUNE

Check that every layer receives a gradient

```
loss = criterion(model(xb), yb)
loss.backward()
for name, p in model.named_parameters():
    g = "None" if p.grad is None else f"{p.grad.norm():.4f}"
    print(f"{name:11s} {str(tuple(p.shape)):9s} {g}")
```

OUTPUT

```
fc1.weight  (32, 60)  0.5140
fc1.bias    (32,)     0.0632
fc2.weight  (32, 32)  None
fc2.bias    (32,)     None
out.weight  (1, 32)   0.3493
out.bias    (1,)      0.1227
```

Here `fc2` is defined in `__init__` but never called in `forward()`, so it can never learn.

Neural Networks / Day 2 · Training neural networks reliably 12

After one backward pass every trainable parameter should have a gradient of reasonable size. This loop prints the gradient norm for each named parameter. The output is real (torch seed 42, the Lab 1 batch from the previous check): the model class defines fc1, fc2 and out, but its forward method goes straight from fc1 through a ReLU to out. PyTorch does not complain; fc2 simply receives no gradient, and its grad stays None.

Three patterns to recognize. None: no gradient reached the parameter, because it was not used in the forward pass, it was frozen with requires_grad=False, or the path to the loss was cut, for example by .detach(), .item() or a NumPy round trip inside forward. Exactly zero: the path exists but carries nothing, for example when every ReLU in a layer is inactive. Tiny but non-zero near the input and larger near the output: vanishing gradients, the subject of Part 2.

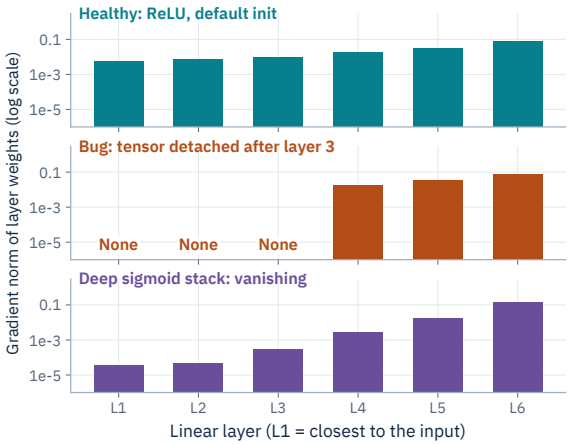
Ask: how would the same bug show up in the one-batch test? The model would still memorize the batch using fc1 and out, so the one-batch test passes. That is why this check exists separately. [3 min]

Missing or zero gradients point to a cut or dead path

DIAGNOSE BEFORE YOU TUNE

Missing or zero gradients point to a cut or dead path

- Compare gradient norms layer by layer after one backward pass
- None: detached, frozen, or unused; exactly zero: dead units
- Tiny near the input, large near the output: vanishing gradients
- Log these norms during training, not only when debugging



Computed: 6-layer MLPs of width 32, one backward pass on 256 random rows, torch seed 0; "None" marks layers that received no gradient.

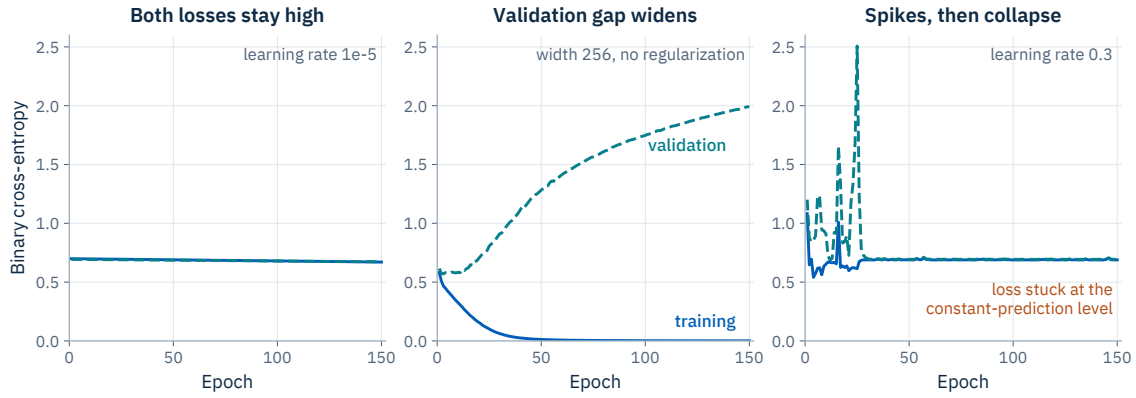
The three panels show the same six-layer network in three states. Top: ReLU with PyTorch's default initialization; every layer receives a gradient, and the largest is only about 13 times the smallest. Middle: a single line x = x.detach() after layer 3 cuts the computational graph, so no gradient reaches layers 1 to 3 at all (their .grad stays None); the network still trains its last three layers, so the loss may even fall, which is what makes this bug dangerous. Bottom: the same stack with sigmoid activations; the gradient reaching layer 1 is more than 4,000 times smaller than at layer 6, because each sigmoid derivative is at most 0.25.

Recommend logging the global gradient norm every step and per-layer norms every few hundred steps; it costs almost nothing and turns silent failures into visible ones. Transition: learning curves are the other cheap signal we always have. [2 min]

Learning curves distinguish several problems

DIAGNOSE BEFORE YOU TUNE

Learning curves distinguish several problems



Computed: synthetic ESP table (seeds 1 and 2), two hidden layers, AdamW, torch seed 0; one setting changed per panel. Similar shapes can have other causes, so treat them as leads, not diagnoses.

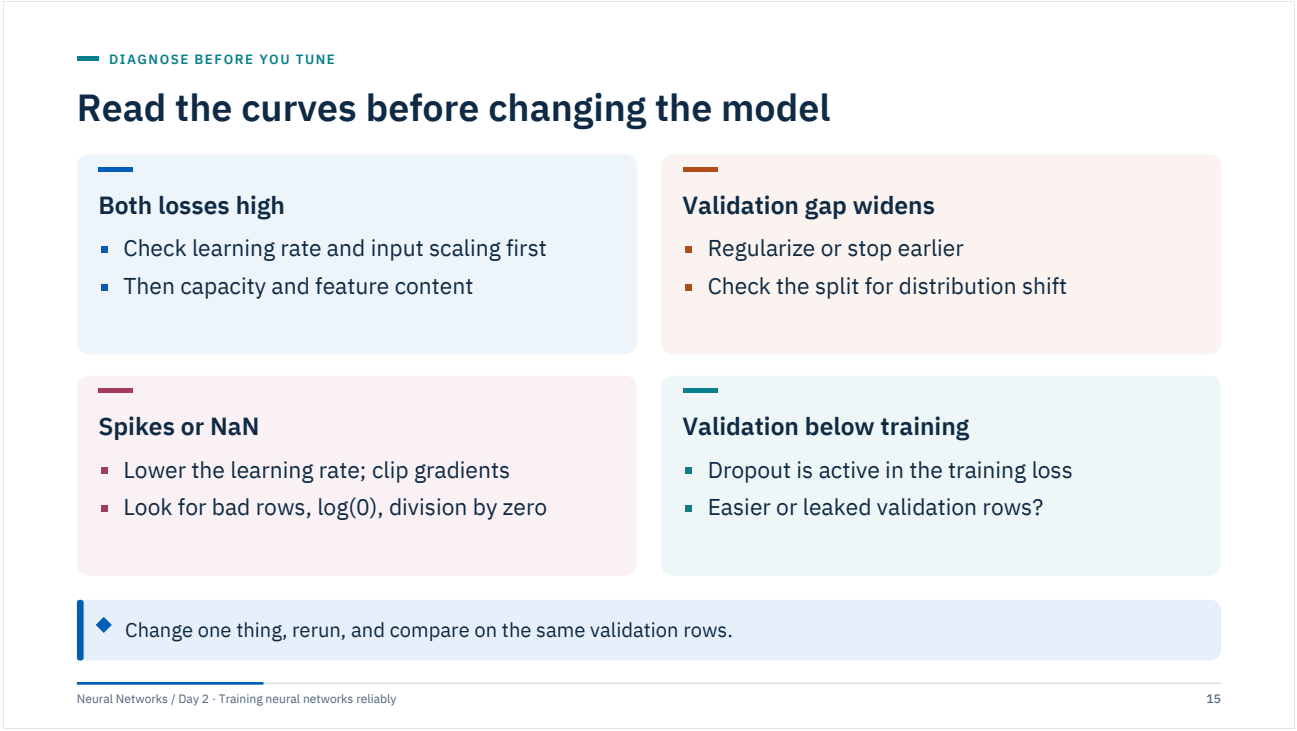
Neural Networks / Day 2 · Training neural networks reliably 14

Three real runs, one change each. Left: a learning rate of 1e-5 is so small that both losses drift from 0.70 to about 0.67 in 150 epochs. The same flat shape can come from too little capacity, uninformative features or a broken pipeline, which is why the earlier checks come first.

Middle: a wide network (256 units) without regularization: training loss goes to zero while validation loss rises from 0.57 to about 2. This is the classic overfitting signature, but a widening gap can also mean the validation rows come from a different distribution, for example a different field or time period.

Right: a learning rate of 0.3 with AdamW. The loss spikes for about 25 epochs and then both curves sit at the constant-prediction level of about 0.69: too-large steps have pushed most ReLU units into a dead state (62 of the 64 units in the second hidden layer never activate on the validation rows), so the network outputs an almost constant value. A flat line at log 2 after a noisy start is a strong hint of this failure. [3 min]

Read the curves before changing the model



These are first moves, not verdicts. Both losses high: before adding capacity, confirm the learning rate is sensible (a range test in Part 4) and inputs are scaled. Validation gap widening: regularization or early stopping, but also ask whether the validation rows are genuinely comparable; grouped or time-ordered splits (covered in ML Module 1) often reveal a real distribution shift rather than overfitting.

Spikes or NaN: usually a learning rate that is too large, occasionally an extreme or corrupted row, or a numerical issue such as taking the log of zero in a hand-written loss. Validation below training surprises people: dropout makes the training loss noisier and higher than the evaluation-mode loss, and the training loss is usually averaged over an epoch while the model improved. If the gap is large, suspect leakage or an easier validation set.

The callout is the discipline: one change per run, same validation rows, so the comparison means something. [2 min]

Debugging from observations

PRACTICE

6 min Pairs

Debugging from observations

Training loss remains flat near 0.69. Predictions are almost constant.

1. List two checks you would run before adding more layers.
2. What experiment would separate a data bug from an optimization problem?
3. The initial loss was 0.69 in one run and 7.5 in another. What does each suggest?

Neural Networks / Day 2 · Training neural networks reliably 16

Give pairs four minutes, then debrief with the next slide. [6 min]

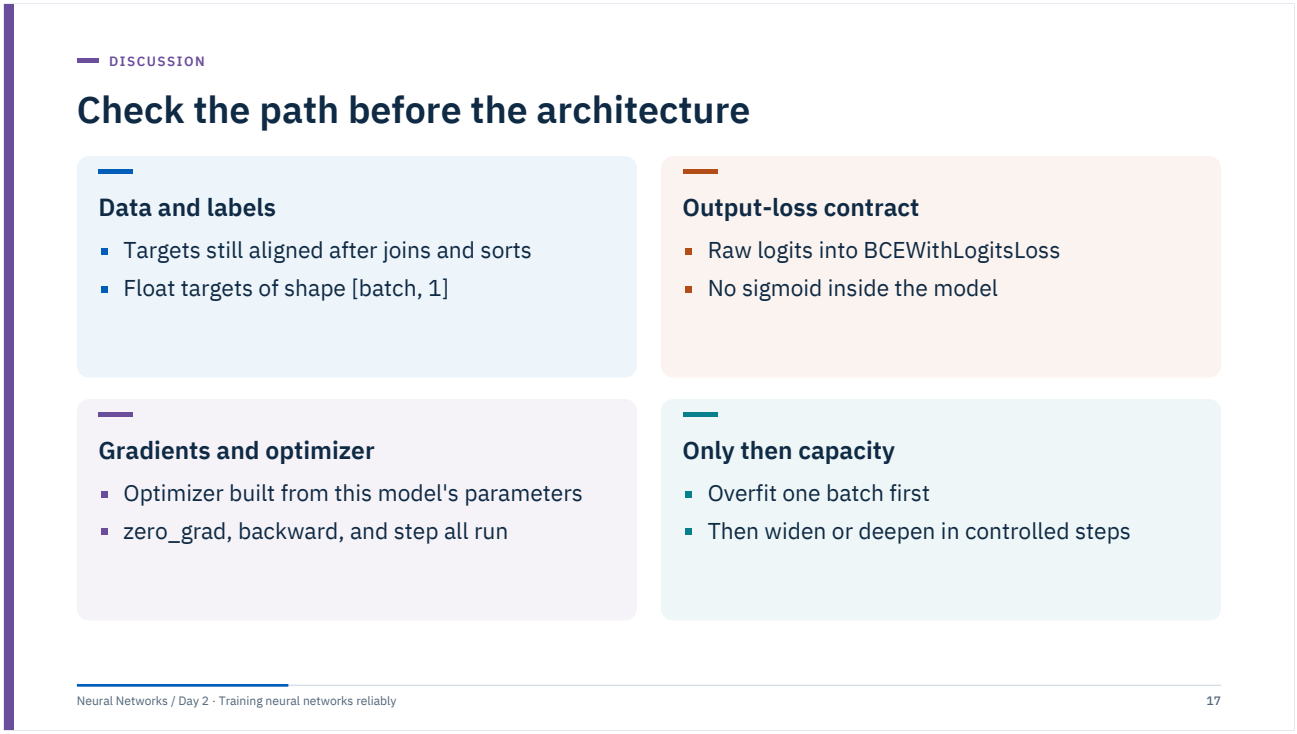
Model answers. Checks before adding layers: target alignment and dtype (are labels still attached to the right rows, are they 0/1 floats of shape [batch, 1]); the output-loss contract (raw logits into BCEWithLogitsLoss, no extra sigmoid); gradients present and non-zero for every layer; the optimizer built from model.parameters() of this model; zero_grad, backward and step all executed; finite values in inputs and loss.

Separating data from optimization: overfit one small, clean batch. If the network memorizes it, the optimization path works and the problem is in the data or the task. If it cannot, the problem is in the model, loss or optimizer wiring. A second experiment: replace the real labels with a simple function of one input column; a working pipeline learns it quickly.

Initial loss: 0.69 equals log 2, so the untrained model is behaving normally and the problem appears during training (learning rate, dead units, or genuinely uninformative features). 7.5 means very confident wrong predictions before training: unscaled inputs, a bad initialization or a label-encoding bug.

Constant predictions can also be rational: if the features carry no information, predicting the base rate is the best the model can do.

Check the path before the architecture

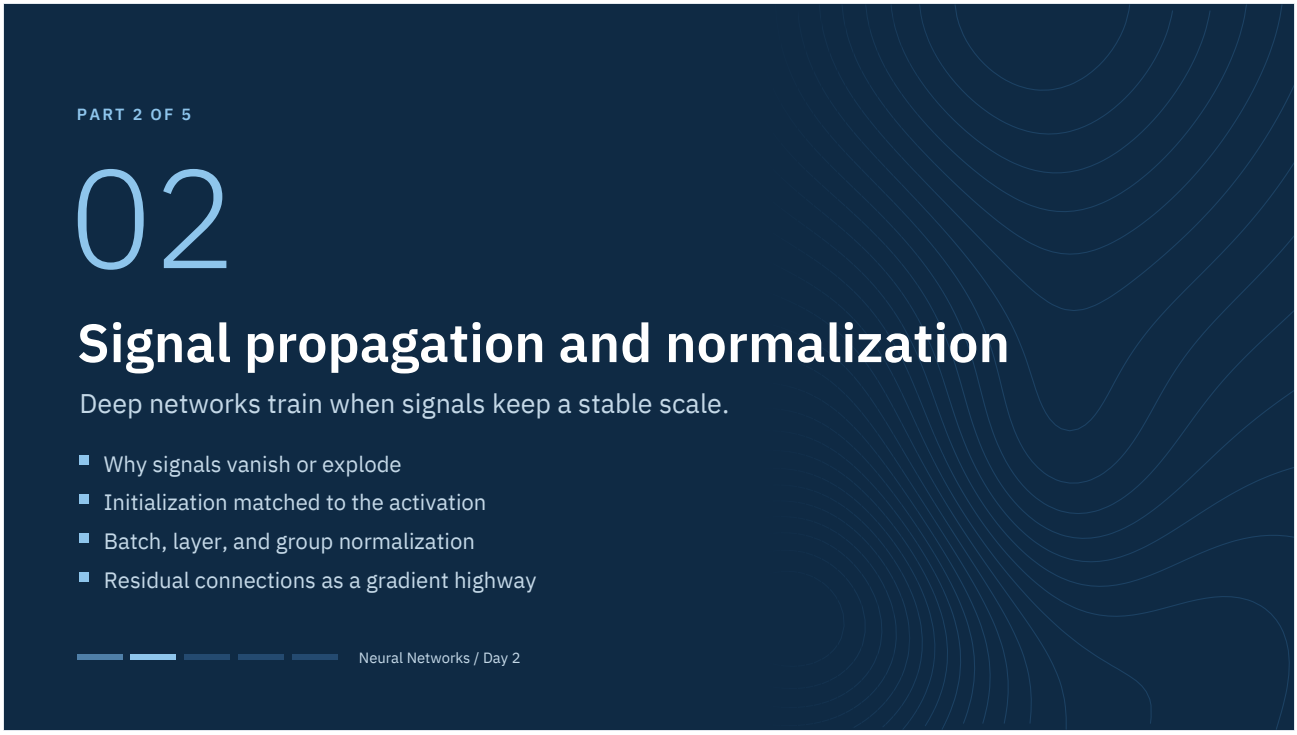


Use this slide to consolidate the pairs' answers. Ask which checks they listed first and which they missed; typically the optimizer wiring (card 3) is forgotten, because it looks too simple to fail.

Stress the order: the cards run left to right, top to bottom, exactly like the flowchart. Adding layers is the last card, and even there the change should be controlled: double the width or add one layer, rerun, and compare on the same validation rows.

If time allows, ask one person to describe a real debugging story from their own work and place it on this grid. It makes the procedure concrete. Then transition: the part after the break explains why the gradient checks sometimes show vanishing values even when nothing is wired wrongly. [3 min]

Signal propagation and normalization



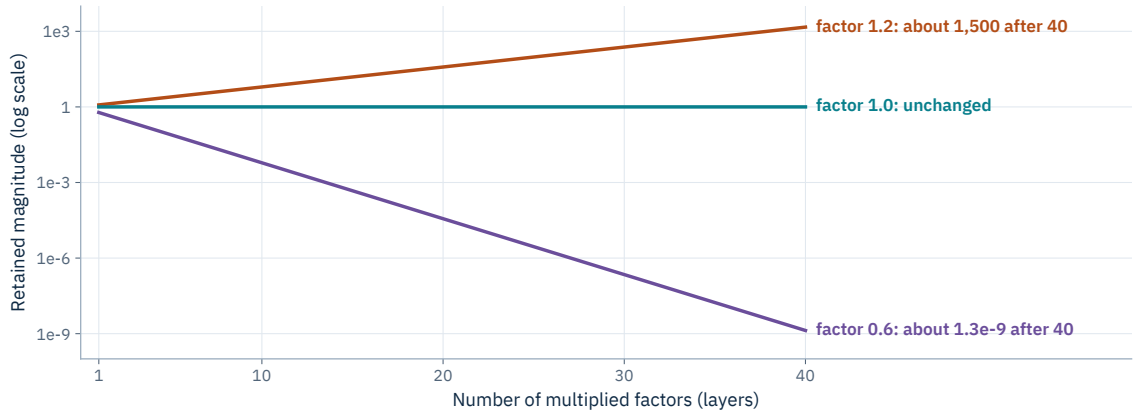
Part 2 runs from about 09:50 to 10:35. The theme is scale: every layer multiplies the forward signal and the backward gradient by something, and in a deep network those multiplications compound. Initialization chooses the starting scale, normalization re-centres and re-scales during training, and residual connections give gradients a path that does not multiply at all.

The ideas carry straight into Day 4 (convolutional networks with batch normalization and residual blocks) and Day 5 (layer normalization in transformers), so it is worth the time to get the mechanism clear here. [1 min]

Gradient products can shrink or explode

SIGNAL PROPAGATION AND NORMALIZATION

Gradient products can shrink or explode



Computed: analytical products of a repeated per-layer factor. The curves illustrate a mechanism, not measured gradients from a trained network.

Neural Networks / Day 2 · Training neural networks reliably 19

The chain rule multiplies local sensitivities. If each layer contributes a factor of 0.6, forty layers retain 0.6 to the power 40, about 1.3e-9 of the signal; a factor of 1.2 grows to about 1,500. Only a factor near 1 keeps the magnitude stable.

Actual gradients involve matrices, activation patterns and many paths, so these curves are a simplified explanation. But the lesson survives the simplification: in a deep network small per-layer biases in scale compound exponentially.

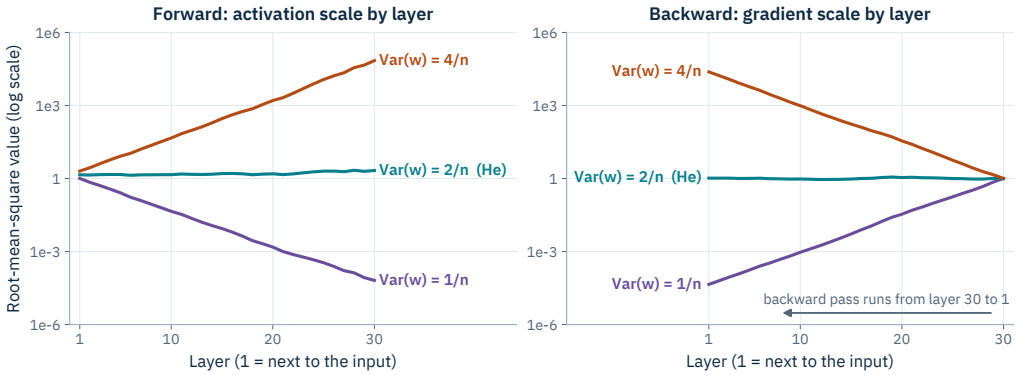
Ask the room: which Day 1 activation function has a derivative that is never larger than 0.25? (Sigmoid.) That is why deep sigmoid stacks were hard to train before better initialization, ReLU-family activations and normalization. [2 min]

Weight scale decides whether signals vanish or explode

SIGNAL PROPAGATION AND NORMALIZATION

Weight scale decides whether signals vanish or explode

Each ReLU layer multiplies the signal's variance by about $n \cdot \mathrm{Var}(w)/2$; over 30 layers small differences compound.



Computed: 30-layer ReLU network, width n = 256, Gaussian weights, no biases or normalization; numpy seed 0.

Neural Networks / Day 2 · Training neural networks reliably 20

Here the mechanism is measured in a real 30-layer ReLU network of width 256. Only the weight variance changes. With Var(w) = 1/n, each layer multiplies the variance of the pre-activations by n × (1/n) × 1/2 = 0.5, because ReLU zeroes half of its inputs; the root-mean-square value falls by a factor of 0.707 per layer, to about 7e-5 at layer 30. With Var(w) = 4/n the factor is 2 and the signal grows to about 7e4. With Var(w) = 2/n, the He initialization, the factor is 1 and the signal stays of order 1 (1.4 at layer 1, 2.1 at layer 30).

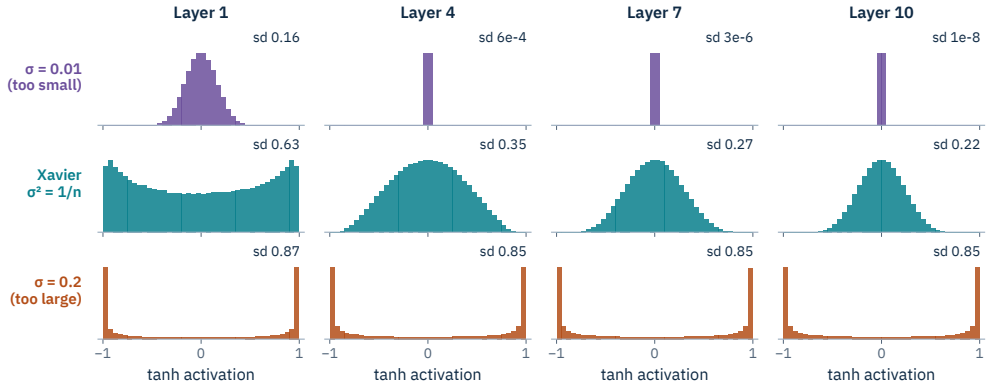
The right panel shows the backward pass. Gradients start at the output (layer 30) and travel back; with the same weights they shrink or grow by the same per-layer factor. So the forward and backward problems come together: the initialization that keeps activations stable also keeps gradients stable. [3 min]

Activation histograms reveal a bad initialization

SIGNAL PROPAGATION AND NORMALIZATION

Activation histograms reveal a bad initialization

A 10-layer tanh network: too small collapses to zero, too large saturates at ±1.



Computed: tanh network of width 256, Gaussian weights with the stated standard deviation, no biases, numpy seed 0; histograms of all activations at four depths.

Neural Networks / Day 2 · Training neural networks reliably 21

Read the rows. Top: weights with standard deviation 0.01. Each layer multiplies the spread by about 0.01 × √256 = 0.16, so by layer 4 the activations have a standard deviation of 6e-4 and by layer 10 about 1e-8: every unit outputs essentially zero, and the gradients with respect to the weights, which are proportional to these activations, vanish too.

Bottom: standard deviation 0.2. The weighted sums are large and tanh saturates: by layer 10 about 60% of the activations lie beyond ±0.9 and a third beyond ±0.99, where the derivative of tanh is below 0.2 and 0.02. The network has a large signal but little gradient.

Middle: Xavier initialization, variance 1/n for this square layer. The distribution stays spread out, slowly narrowing from 0.63 at layer 1 to 0.22 at layer 10; units are in their sensitive range. This is the picture to look for when you plot histograms from your own model with forward hooks. [3 min]

Record activation statistics with forward hooks

SIGNAL PROPAGATION AND NORMALIZATION

Record activation statistics with forward hooks

```
stats = {}
def record(name):
    def hook(module, inputs, output):
        stats[name] = output.std().item()
    return hook

for name, m in model.named_modules():
    if isinstance(m, nn.Tanh):
        m.register_forward_hook(record(name))
with torch.no_grad():
    model(xb)
print({k: f"{v:.1e}" for k, v in stats.items()})
```

OUTPUT

```
{'1': '1.6e-01', '3': '2.5e-02', '5': '4.0e-03', '7': '6.4e-04'}
```

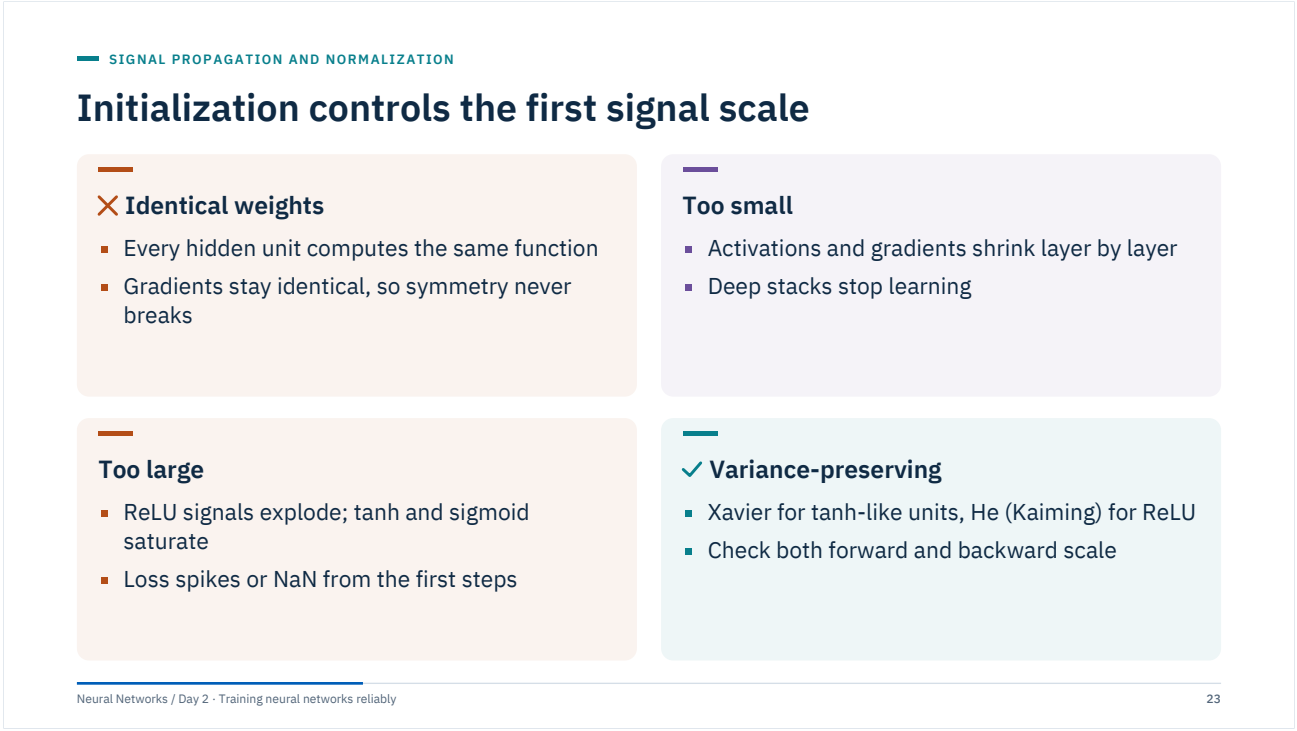
Four tanh layers of width 256, weights drawn with standard deviation 0.01, zero biases: the spread shrinks about 6× per layer.

A forward hook is a function PyTorch calls every time a module produces an output. Here it stores the standard deviation of each tanh layer's output, without changing the model code. The output is from a real four-layer tanh network of width 256 whose weights were drawn with standard deviation 0.01 and whose biases were set to zero (PyTorch's default biases would otherwise dominate such tiny signals): each layer shrinks the spread by about 0.16, as on the previous slide.

In practice you would store histograms or percentiles as well and log them every few hundred steps. Hooks can also be registered on gradients (register_full_backward_hook on modules, or tensor.register_hook) to track backward scale.

Two cautions: remove hooks you no longer need (register_forward_hook returns a handle with a remove() method), and use torch.no_grad() or inference mode when you only measure. [2 min] @source https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

Initialization controls the first signal scale



Symmetry first: if all weights in a layer start equal, every unit in that layer receives the same gradient and stays identical forever, so a layer of 64 units behaves like one unit. Random initialization breaks the symmetry. Zero biases are usually fine; it is identical weights that cause the problem.

Too small and too large are the two failure modes from the histograms. Variance-preserving schemes choose the scale so that the variance of activations (and gradients) stays roughly constant from layer to layer: Xavier (Glorot and Bengio, 2010) for tanh and sigmoid, He or Kaiming (He et al., 2015) for ReLU.

Practical advice: use the framework's documented initializer for the chosen layer and activation as the baseline, and only change it with evidence from histograms or gradient norms. [2 min] @source https://proceedings.mlr.press/v9/glorot10a.html ; https://arxiv.org/abs/1502.01852

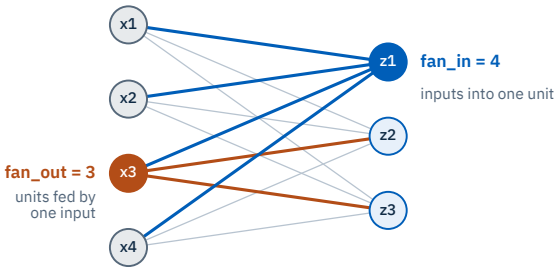
Initialization variance scales with fan-in

SIGNAL PROPAGATION AND NORMALIZATION

Initialization variance scales with fan-in

$$\mathrm{Var}(w) \approx \frac{2}{\mathrm{fan}_{\mathrm{in}} + \mathrm{fan}_{\mathrm{out}}} \quad \text{or} \quad \frac{2}{\mathrm{fan}_{\mathrm{in}}}$$

- Xavier (Glorot) averages fan-in and fan-out: suits tanh and sigmoid
- He (Kaiming) uses 2 / fan-in: compensates for ReLU zeroing half its inputs
- Distribution (normal or uniform) and gain change the exact bounds



$\mathrm{Var}(z) = \mathrm{fan}_{\mathrm{in}} \cdot \mathrm{Var}(w) \cdot E[x^2]$

keep Var(z) steady → Var(w) ≈ 1 / fan_in (× 2 for ReLU)

Schematic: one dense layer with fan-in 4 and fan-out 3.

Neural Networks / Day 2 · Training neural networks reliably 24

Define fan-in as the number of inputs feeding one unit and fan-out as the number of units fed by one input. For z = sum of w times x with independent zero-mean weights, Var(z) = fan_in × Var(w) × E[x²]. Keeping Var(z) near the variance of the inputs therefore requires Var(w) near 1/fan_in. Xavier uses 2/(fan_in + fan_out) to balance the forward and backward passes. He initialization uses 2/fan_in because a ReLU sets half of its inputs to zero, halving E[x²].

This is a concise variance reference, not interchangeable parameter settings for every initializer. PyTorch's nn.Linear default draws from a uniform distribution with variance 1/(3·fan_in); torch.nn.init.kaiming_normal_ and xavier_uniform_ expose mode and gain parameters. Check the weight orientation when applying an initializer manually: nn.Linear stores weights as [out_features, in_features]. [3 min] @source https://docs.pytorch.org/docs/stable/nn.init.html

How fast does a bad scale compound?

PRACTICE

5 min · Individual

How fast does a bad scale compound?

A 10-layer ReLU network has fan-in 256 in every layer; weights are Gaussian with standard deviation σ.

1. Compute the per-layer variance factor $256\,\sigma^2/2$ for σ = 0.01, σ = 0.1, and He initialization.
2. By what factor has the activation standard deviation changed after 10 layers in each case?
3. Which σ does He initialization give here?

Neural Networks / Day 2 · Training neural networks reliably 25

Let individuals work for three minutes, then reveal. [5 min]

The per-layer variance factor for ReLU is fan_in × σ² / 2.

σ = 0.01: 256 × 0.0001 / 2 = 0.0128. The standard deviation changes by √0.0128 = 0.113 per layer; after 10 layers the variance factor is 0.0128^10, so the standard deviation factor is 0.0128^5 = 3.4e-10. The signal has vanished.

σ = 0.1: 256 × 0.01 / 2 = 1.28 per layer; after 10 layers the standard deviation factor is 1.28^5 = 3.44. It grows by a factor of about 3.4, and a deeper network would keep growing it.

He: σ² = 2/256, so σ = √(2/256) = 0.0884 and the factor is exactly 1: the scale is preserved.

Common errors: forgetting the factor 1/2 for ReLU, and multiplying standard deviations instead of variances (the per-layer standard deviation factor is the square root of the variance factor). Note the pleasing symmetry: 0.0128 and 1.28 differ by a factor of 100, and σ = 0.01 and 0.1 differ by 10.

Normalization uses different axes

SIGNAL PROPAGATION AND NORMALIZATION

Normalization uses different axes

Method	Statistics used	Training and inference issue
Input standardization	Training-set feature statistics	Keep fitted values fixed
Batch normalization	Batch statistics during training	Running estimates during evaluation
Layer normalization	Features within each example	No batch running-average dependency
Group normalization	Channel groups within each example	Independent of batch size

Batch, layer, and group normalization all add a learned scale and shift after normalizing.

Normalization layers re-centre and re-scale activations during training, which keeps later layers in a good range even as earlier weights change. The table compares where the statistics come from.

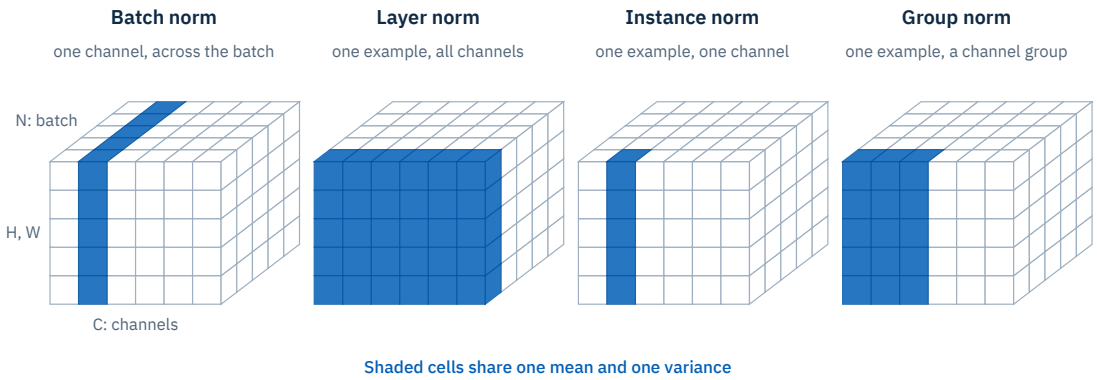
Input standardization is the preprocessing step used in the Day 1 pipelines (covered in ML Module 1): mean and standard deviation fitted on the training rows only and then frozen. Batch normalization computes statistics over the current mini-batch during training and switches to running estimates in evaluation mode. Layer normalization uses the features of each example, so training and inference behave identically and batch size does not matter; it is common in transformers (Day 5). Group normalization is the same idea over groups of channels, designed for convolutional networks trained with small batches.

BatchNorm, LayerNorm and GroupNorm include a learnable scale and shift after normalizing (by default in PyTorch). None of them removes the need for sensible input preprocessing. [2 min] @source https://arxiv.org/abs/1502.03167 ; https://arxiv.org/abs/1607.06450 ; https://arxiv.org/abs/1803.08494

Batch, layer, and group norm share statistics differently

SIGNAL PROPAGATION AND NORMALIZATION

Batch, layer, and group norm share statistics differently



Shaded cells share one mean and one variance

Schematic after Wu and He (2018). For a dense layer H, W = 1: batch norm normalizes each feature over the batch; layer norm normalizes each example over its features.

Each cube is one layer's activations for a mini-batch: N examples along the depth axis, C channels or features across, and the spatial positions H and W stacked vertically (for images or seismic patches). The shaded cells are the values that share one mean and one variance.

Batch norm: one channel, all examples, all positions. Layer norm: one example, all channels and positions. Instance norm: one example, one channel. Group norm: one example, a group of channels; with one group it equals layer norm, with one channel per group it equals instance norm.

For the tabular and sensor models in this course, H and W are 1, so each cube flattens into a table of examples by features: batch norm normalizes columns, layer norm normalizes rows. Ask: which of these can compute its statistics from a single example at inference time? All except batch norm, which is why batch norm needs running averages. [2 min] @source https://arxiv.org/abs/1803.08494

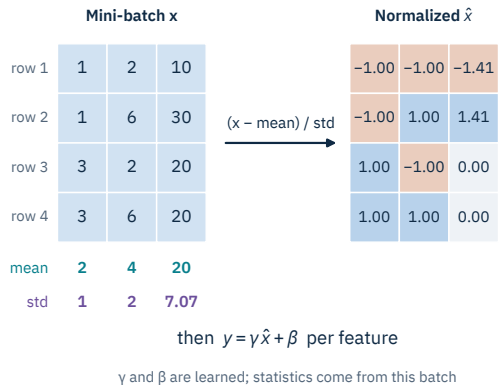
Batch normalization includes learned scale and shift

SIGNAL PROPAGATION AND NORMALIZATION

Batch normalization includes learned scale and shift

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta$$

- μ_B and σ_B^2 are computed over the batch for each feature
- γ and β are learned, so the layer can undo the scaling
- Evaluation uses running estimates instead



Exact arithmetic with torch.nn.BatchNorm1d in training mode (affine off); population standard deviation over the batch.

Neural Networks / Day 2 · Training neural networks reliably 28

Walk through the numbers. Feature 1 has values 1, 1, 3, 3: mean 2, standard deviation 1, so the normalized values are -1, -1, 1, 1. Feature 2 (2, 6, 2, 6): mean 4, standard deviation 2. Feature 3 (10, 30, 20, 20): mean 20, standard deviation √50 = 7.07, giving -1.41, 1.41, 0 and 0. The figure was computed with PyTorch, which uses the population (biased) variance over the batch for normalization in training mode; ε = 1e-5 prevents division by zero.

Then y = γ x̂ + β with γ and β learned per feature, initialized to 1 and 0. If the best scale for the next layer is not mean 0 and variance 1, the network can learn it.

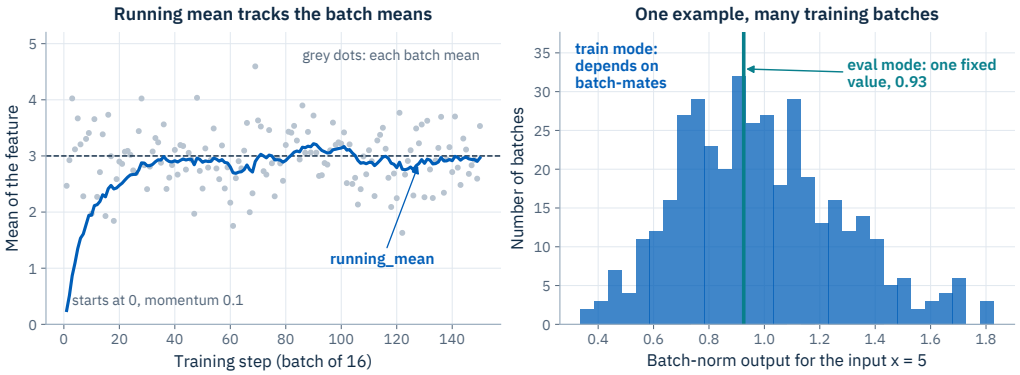
State which axes a layer normalizes for dense versus convolutional data. Avoid the blanket claims that batch normalization prevents overfitting or fixes every gradient problem; its benefits depend on architecture and batch behaviour. [3 min] @source https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html

BatchNorm uses batch statistics only in training mode

SIGNAL PROPAGATION AND NORMALIZATION

BatchNorm uses batch statistics only in training mode

Running averages are updated in training and used after `model.eval()`; forget `eval()` and predictions depend on batch-mates.



Computed with torch.nn.BatchNorm1d (momentum 0.1) on a synthetic feature with mean 3 and standard deviation 2; batches of 16; numpy seed 0.

Neural Networks / Day 2 · Training neural networks reliably 29

Left: during training, each batch has its own mean (grey dots). BatchNorm keeps a running average, updated as running = (1 - momentum) × running + momentum × batch value, with momentum 0.1 in PyTorch. It starts at 0 and converges to the true mean of 3 within about 30 steps; the running variance is tracked the same way.

Right: the same input value, x = 5, placed in 400 different training batches. In training mode its normalized output ranges from about 0.35 to 1.8 depending on the other 15 rows in the batch. In evaluation mode it is always 0.93, computed with the running statistics.

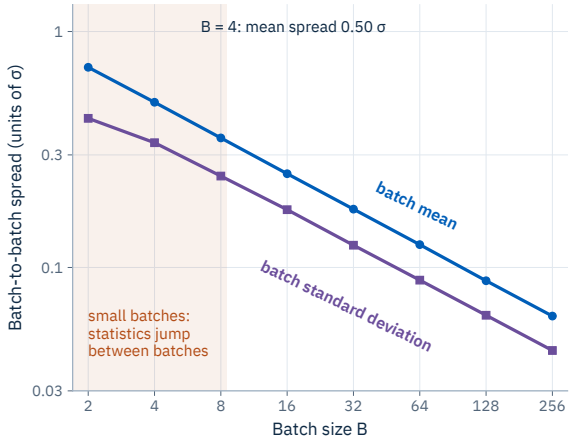
Operational consequence: if you forget model.eval() before inference, the prediction for one well-log sample depends on which other samples happen to share its batch, and a batch of one row may fail outright. Small or unrepresentative batches also make the training statistics noisy. [3 min]

Batch statistics are noisy when batches are small

SIGNAL PROPAGATION AND NORMALIZATION

Batch statistics are noisy when batches are small

- Batch norm needs large, representative batches (common in Day 4 CNNs)
- With 2 to 8 rows per batch, the mean and variance jump between batches
- Layer and group norm use one example: fine for small batches and Day 5 sequences
- None of them replaces input standardization fitted on training rows



Computed: 20,000 random batches per size from a standard-normal feature, numpy seed 0; spread across batches in units of σ.

Neural Networks / Day 2 · Training neural networks reliably 30

The figure quantifies why batch normalization prefers large batches. For a feature with standard deviation σ, the batch mean varies from batch to batch with a spread of σ divided by the square root of the batch size: 0.71 σ at B = 2, 0.50 σ at B = 4, 0.25 σ at B = 16 and 0.125 σ at B = 64. The batch standard deviation is noisy in the same way (0.34 σ at B = 4, 0.09 σ at B = 64). With tiny batches, every training step normalizes with a noticeably different mean and scale; in PyTorch a training batch of one row fails outright with "Expected more than 1 value per channel when training".

Decision rule: batch normalization suits batches of tens of examples or more that resemble the inference data, as in many image models (Day 4). It struggles with very small batches, such as large 3-D seismic volumes, and with variable-length sequences. Layer and group normalization use per-example statistics and behave the same in training and inference (Day 5 transformers).

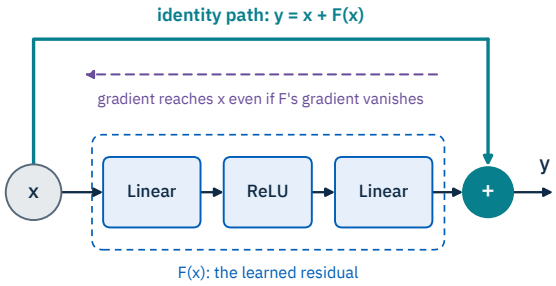
For today's small tabular models, standardized inputs and a sensible initialization are often enough; add normalization when depth or instability calls for it. [2 min]

Residual connections give gradients a highway

SIGNAL PROPAGATION AND NORMALIZATION

Residual connections give gradients a highway

- The block learns a correction $F(x)$ added to its input
- The identity term gives early layers a direct gradient path
- Very deep networks become trainable; Day 4's ResNets build on this
- x and $F(x)$ must have the same shape, or a projection is added



$$\partial y/\partial x = I + \partial F/\partial x$$

Day 4 uses this block in residual CNNs

Schematic residual block: y = x + F(x).

A residual (skip) connection adds the block's input to its output: y = x + F(x). The block only has to learn the correction F, and if the best thing to do is nothing, it can learn F close to zero.

The gradient consequence is in the derivative: dy/dx = I + dF/dx. Even if the gradient through F is tiny, the identity term passes the upstream gradient straight to x. Stack thirty such blocks and there is still a path from the loss to the first block that involves no multiplications by small factors.

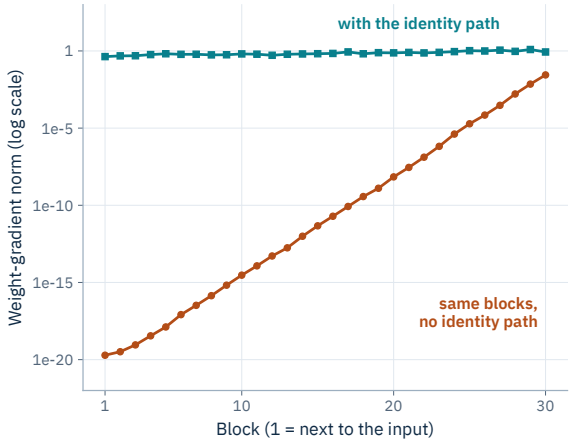
He et al. (2016) used this to train networks with more than a hundred layers; Day 4 shows the convolutional version, and the transformer blocks in Day 5 use the same idea with layer normalization. When the shape changes, for example more channels, the skip path uses a small projection layer so that x and F(x) can be added. [2 min] @source https://arxiv.org/abs/1512.03385

Skip paths keep gradients alive in deep stacks

SIGNAL PROPAGATION AND NORMALIZATION

Skip paths keep gradients alive in deep stacks

- Same 30 blocks and identical weights; only the identity path differs
- Without it, the first block's gradient is about 10^{-20}
- With it, every block receives a gradient of similar size



Computed: 30 Linear-ReLU-Linear blocks of width 64, PyTorch default initialization, one backward pass on 256 random rows; torch seeds 0 and 1.

This is the mechanism measured. Both networks have the same thirty blocks, each Linear, ReLU, Linear, with identical weights from the same seed. The only difference is whether each block adds its input back.

Without the identity path, the gradient norm reaching block 30 is about 0.03 and block 1 receives about 2e-20: with PyTorch's default initialization each block shrinks the signal, and thirty blocks compound it. Those early blocks will not learn in any practical number of steps.

With the identity path, every block receives a gradient between about 0.4 and 1.2. Nothing else changed: no new initialization, no normalization.

Caveat for the notes: residual networks without any normalization can let the forward signal grow with depth; real architectures combine skip connections with normalization or scaled initialization. [2 min]

Which fix would you try first?

PRACTICE

5 min Pairs

Which fix would you try first?

For each observation, name the first change you would test and the evidence that would confirm it.

1. Activation histograms collapse to a spike at zero by layer 6.
2. A BatchNorm model gives different predictions for the same well-log sample depending on its batch.
3. A 40-layer plain MLP trains no better than a 4-layer one.

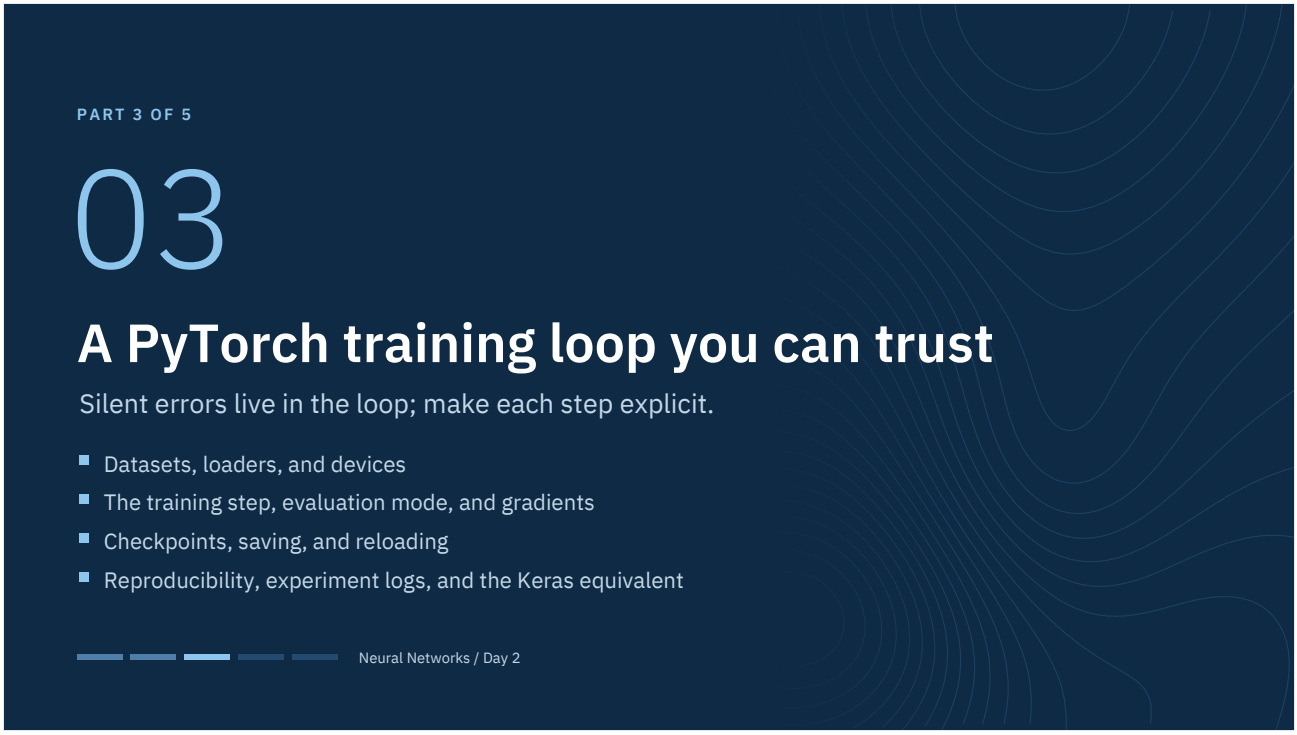
Three minutes in pairs, two minutes to debrief. [5 min]

1. Collapse to zero: the weight scale is too small for the activation (or the inputs are tiny). Switch to the initializer matched to the activation (He for ReLU, Xavier for tanh) and check that the input features are standardized. Evidence: the forward-hook statistics stay roughly constant across layers and gradient norms reach the first layers.

2. Batch-dependent predictions: the model is being run in training mode at inference, so batch statistics are used. Call model.eval() before predicting. Evidence: the same sample gives the same output alone or in any batch. If training batches are tiny, consider layer or group normalization instead.

3. Deep plain MLP no better than shallow: likely vanishing gradients in the early layers, or simply no benefit from depth for this problem. Add residual connections and normalization, and check per-layer gradient norms. Evidence: early-layer gradients comparable to late ones and a lower training loss. Also compare against the 4-layer model on validation; depth is not automatically useful for tabular data.

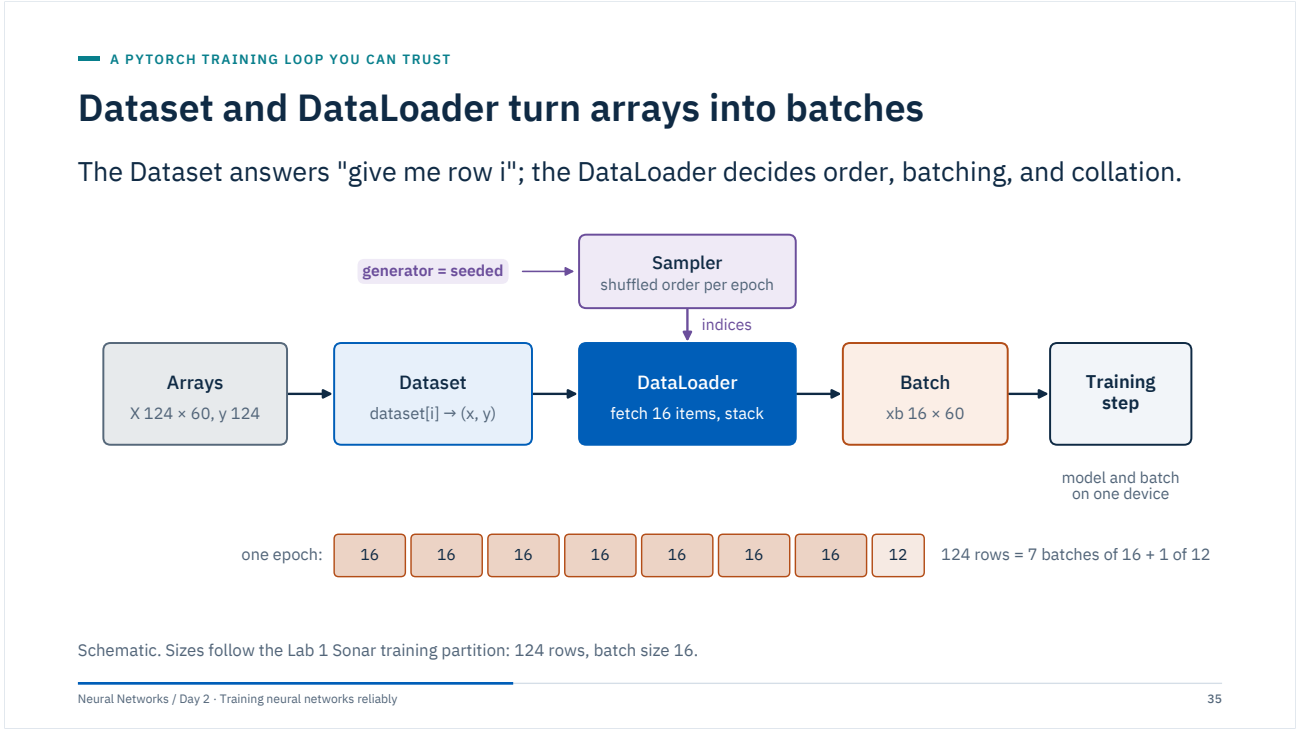
A PyTorch training loop you can trust



Part 3 runs from about 10:50 to 11:30, straight into Lab 1. Everything here appears in the Lab 1 notebook, so the aim is recognition: when learners open the notebook they should be able to point at each piece and say what it is for.

Use the PyTorch route as the main code path, then translate to Keras at the end. Framework choice should follow existing systems, deployment needs and team skills; avoid unsupported blanket statements about one framework being faster or better. [1 min]

Dataset and DataLoader turn arrays into batches



Walk left to right. The arrays are the scaled training features and targets. A Dataset is any object with a length and an indexing rule that returns one example; TensorDataset wraps tensors that are already in memory, and a custom Dataset class can read files or database rows lazily.

The DataLoader asks a sampler for indices (shuffled once per epoch when shuffle=True), fetches that many items from the dataset and stacks them into batch tensors. With 124 rows and a batch size of 16 an epoch has 8 batches: seven of 16 and a last one of 12. You can drop the short batch with drop_last=True, which matters for batch normalization with tiny final batches.

The sampler's randomness comes from a generator; passing a seeded generator makes the batch order repeatable, which is the first step towards reproducible runs. [2 min] @source https://docs.pytorch.org/docs/stable/data.html

Build loaders with an explicit, seeded shuffle

A PYTORCH TRAINING LOOP YOU CAN TRUST

Build loaders with an explicit, seeded shuffle

```
from torch.utils.data import (
    TensorDataset, DataLoader)

train_ds = TensorDataset(Xt, yt)
g = torch.Generator().manual_seed(42)
train_loader = DataLoader(
    train_ds, batch_size=16,
    shuffle=True, generator=g)
val_loader = DataLoader(
    TensorDataset(Xv, yv),
    batch_size=64, shuffle=False)
xb, yb = next(iter(train_loader))
print(xb.shape, yb.shape, len(train_loader))
```

OUTPUT

```
torch.Size([16, 60]) torch.Size([16, 1]) 8
```

- Shuffle training batches, never validation
- A seeded generator repeats the batch order
- `len(loader)` is the number of batches per epoch

Neural Networks / Day 2 · Training neural networks reliably 36

This code is the Lab 1 loader setup. Xt and yt are the standardized training features and float targets of shape [124, 60] and [124, 1]; Xv and yv are the validation equivalents.

The printed output is real: one batch holds 16 rows of 60 features and a target column of shape [16, 1], and the loader yields 8 batches per epoch. Keeping the target as [batch, 1] matches the model's output shape for BCEWithLogitsLoss; if you prefer one-dimensional targets, squeeze the logits instead, but be consistent.

Validation data are not shuffled because order does not matter for evaluation and a fixed order makes debugging easier. The validation batch size can be larger because no gradients are stored. For reproducibility with multiple worker processes, also set worker_init_fn as described in the PyTorch randomness notes. [2 min] @source https://docs.pytorch.org/docs/stable/notes/randomness.html

Model and batches must share one device

A PYTORCH TRAINING LOOP YOU CAN TRUST

Model and batches must share one device

```python
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

model = model.to(device)   # before the optimizer
for xb, yb in train_loader:
    xb = xb.to(device)
    yb = yb.to(device)
```

- Move the model once, before building the optimizer
- Move every batch inside the loop
- Use `.cpu()` before converting results to NumPy
- Small tabular models are often fastest on CPU

Neural Networks / Day 2 · Training neural networks reliably 37

PyTorch never moves tensors silently. The model's parameters and every input must be on the same device, otherwise the forward pass raises a runtime error about tensors on different devices. The code picks a CUDA GPU if present, then Apple's MPS backend, then the CPU.

Move the model before constructing the optimizer: the PyTorch documentation asks for this so that the optimizer holds references to the parameters on the right device. Batches are moved inside the loop, one at a time. Results must come back to the CPU before converting to NumPy for plotting or scikit-learn metrics.

For the small tabular problems in this course, a GPU may be slower than the CPU because transfer and launch overheads dominate; measure run time rather than assuming. The lab notebook fixes the device to CPU and one thread for reproducibility on classroom laptops. [2 min] @source https://docs.pytorch.org/docs/stable/optim.html

A small binary model in PyTorch

A PYTORCH TRAINING LOOP YOU CAN TRUST

A small binary model in PyTorch

```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(n_features, 32), nn.ReLU(),
    nn.Dropout(0.2), nn.Linear(32, 1)
)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3, weight_decay=1e-3
)
```

The final layer emits one raw logit per row; the loss applies the sigmoid internally.

Neural Networks / Day 2 · Training neural networks reliably 38

n_features comes from the fitted numeric schema, 60 for Sonar. The final layer emits one raw logit; BCEWithLogitsLoss combines the sigmoid and the binary cross-entropy in a numerically stable way, which is why there is no Sigmoid layer in the model (recall the floor in the one-batch test).

Ask learners to count the parameters for n_features = 60: 60 × 32 + 32 = 1,952 in the first layer, 32 + 1 = 33 in the output layer, 1,985 in total. Dropout has no parameters.

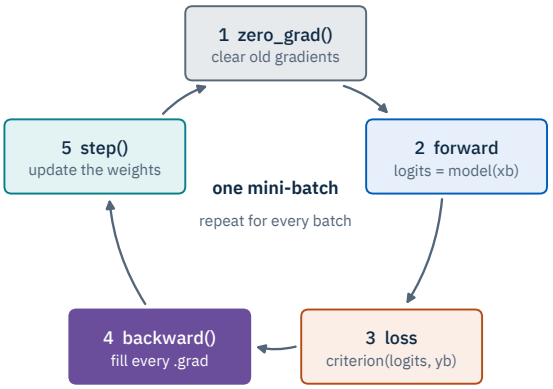
Learning rate, width, dropout and decay are the Lab 1 settings: illustrative starting points to validate, not tuned results for the supplied data. AdamW is used because its weight decay is decoupled from the adaptive step; Part 4 explains the difference from Adam with an L2 penalty. [2 min]

Every training step repeats five operations

A PYTORCH TRAINING LOOP YOU CAN TRUST

Every training step repeats five operations

- `zero_grad` clears gradients left from the previous batch
- Forward and loss build the graph on the current batch
- `backward` fills `.grad`; `step` applies the optimizer rule
- Skip or reorder one and training fails quietly



Schematic of one PyTorch optimization step.

Neural Networks / Day 2 · Training neural networks reliably 39

PyTorch accumulates gradients: each backward() call adds into the .grad field of every parameter. That is useful for gradient accumulation across several small batches, but it means you must clear the gradients before each new batch, which is what zero_grad does (set_to_none=True frees the memory instead of writing zeros).

The forward pass computes logits and the loss builds the computational graph; backward walks the graph and fills .grad; step reads .grad and changes the weights according to the optimizer's rule.

Ask: what happens if zero_grad is forgotten? The gradient of batch k contains the sum of all previous gradients, so updates grow and become erratic, a behaviour that can look like a learning-rate problem. What happens if step is called before backward? The optimizer uses stale or empty gradients. Neither raises an error. [2 min]

The training loop orders zero_grad, backward, and step

A PYTORCH TRAINING LOOP YOU CAN TRUST

The training loop orders zero_grad, backward, and step

```
model.train()
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(xb)                      # (B, 1)
    loss = criterion(logits, yb)            # yb: (B, 1) floats
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
```

Clipping is optional; if used, it belongs after `backward()` and before `step()`.

Neural Networks / Day 2 · Training neural networks reliably 40

Move the model to the device before the loop. model.train() switches dropout and batch normalization to training behaviour; it must be called again at the start of every epoch because the evaluation code switches the model to eval mode.

The shapes are part of the contract: logits of shape [B, 1] and float targets of shape [B, 1]. A classic error is comparing [B, 1] outputs with [B] targets under MSELoss, which broadcasts to a [B, B] matrix with only a warning; BCEWithLogitsLoss raises an error instead.

Clipping is shown to illustrate where it belongs: after backward, before the optimizer step. It rescales the whole gradient when its norm exceeds 1.0. It controls rare large updates but does not solve vanishing gradients. [2 min] @source https://docs.pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html

Evaluation mode and gradient tracking are separate

A PYTORCH TRAINING LOOP YOU CAN TRUST

Evaluation mode and gradient tracking are separate

```
model.eval()
with torch.inference_mode():
    logits = model(X_val.to(device))                  # (n, 1)
    val_loss = criterion(logits, y_val.to(device))    # y_val: (n, 1)
    probabilities = torch.sigmoid(logits).squeeze(-1).cpu().numpy()
```

- `eval()` changes dropout and normalization behaviour
- `inference_mode()` switches off autograd bookkeeping for inference

Neural Networks / Day 2 · Training neural networks reliably 41

Two separate switches. model.eval() changes how certain layers compute: dropout passes activations through unchanged and batch normalization uses its running statistics. It does not stop PyTorch from recording operations for autograd. torch.inference_mode() (or torch.no_grad()) stops the recording, which saves memory and time, but does not change layer behaviour.

For large validation sets, use a DataLoader and aggregate the loss weighted by the number of rows in each batch; compute ranking metrics such as ROC AUC across all predictions rather than averaging per-batch values. The probabilities are squeezed to one dimension and moved to the CPU for NumPy and scikit-learn.

Ask: what goes wrong if you use no_grad but forget eval? Dropout stays active, so predictions are noisy and systematically different from the model you selected. [2 min] @source https://docs.pytorch.org/docs/stable/notes/autograd.html

Mode switches and autograd switches do different jobs

A PYTORCH TRAINING LOOP YOU CAN TRUST

Mode switches and autograd switches do different jobs

Call	Dropout	BatchNorm statistics	Gradient tracking
`train()`	active	batch; running averages updated	no effect
`eval()`	off	running averages used	no effect
`no_grad()`	no effect	no effect	off
`inference_mode()`	no effect	no effect	off; stricter, faster

Evaluation needs both: `eval()` for layer behaviour, and a no-grad context for speed and memory.

Use this table as the reference card for the lab. The first two rows change module behaviour and nothing else; the last two change autograd and nothing else.

inference_mode is a stricter version of no_grad: tensors created inside it cannot later be used in autograd, which lets PyTorch skip more bookkeeping. Use no_grad when you need to compute something without gradients but might reuse the tensors in training code, and inference_mode for pure prediction.

A quick check question: during early-stopping evaluation inside the training loop, which calls do you need? Answer: model.eval() plus a no-grad context before evaluating, and model.train() again before the next training epoch. The Lab 1 notebook does exactly this at every epoch. [2 min]

A reference to state_dict is not a snapshot

A PYTORCH TRAINING LOOP YOU CAN TRUST

A reference to state_dict is not a snapshot

```python
import copy

best_ref = model.state_dict()                    # live references
best_copy = copy.deepcopy(model.state_dict())    # snapshot
w0 = best_copy["0.weight"][0, 0].item()

train_one_epoch(model, train_loader, optimizer)   # weights change

print(f"{w0:.4f}")
print(f"{best_ref['0.weight'][0, 0].item():.4f}")
print(f"{best_copy['0.weight'][0, 0].item():.4f}")
```

OUTPUT

```
0.0987
0.0919
0.0987
```

Neural Networks / Day 2 · Training neural networks reliably 43

This is the most common checkpointing bug, shown with real output from the Lab 1 model (torch seed 42, a fresh seeded loader, AdamW with learning rate 1e-3 and weight decay 1e-3). model.state_dict() returns an ordered dictionary whose tensors share storage with the live parameters. Saving that dictionary in a variable does not freeze anything: after one epoch, best_ref shows the new weight (0.0919) instead of the value at the time it was "saved" (0.0987).

copy.deepcopy creates independent tensors, so best_copy still holds 0.0987. Saving to disk with torch.save also creates an independent copy.

train_one_epoch is the helper from the lab: model.train(), then zero_grad, forward, loss, backward and step for each batch.

Consequence for early stopping: if you keep a reference instead of a copy, "restoring the best checkpoint" silently restores the last epoch, and your reported validation score does not belong to the model you ship. [2 min]

Early stopping needs a copied checkpoint

A PYTORCH TRAINING LOOP YOU CAN TRUST

Early stopping needs a copied checkpoint

```
import copy

best_loss, stale, patience = float("inf"), 0, 10
for epoch in range(1, max_epochs + 1):
    train_one_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)       # eval mode inside
    if val_loss < best_loss:
        best_loss, best_epoch, stale = val_loss, epoch, 0
        best_state = copy.deepcopy(model.state_dict())
    else:
        stale += 1
        if stale >= patience:
            break

model.load_state_dict(best_state)                # restore, then
model.eval()                                     # evaluate or save
```

Neural Networks / Day 2 · Training neural networks reliably 44

This is the full early-stopping pattern. Initialize best_loss to infinity and stale to zero, and choose patience before training. After each epoch evaluate on the validation rows in eval mode; if the loss improved, record the epoch and copy the state; otherwise count a stale epoch and stop once the count reaches the patience.

After the loop, restore the selected checkpoint before any evaluation or saving. Without the restore, you would evaluate the model from the last epoch, which is up to patience epochs past the best one.

Two refinements used in the lab: a minimum improvement (the notebook requires the loss to drop by more than 1e-7) so that tiny fluctuations do not reset the counter, and saving the best state to disk as well as memory for long runs. evaluate is a helper that switches to eval mode, runs the validation loader under inference mode and returns the row-weighted mean loss. [2 min]

Save the state dict with everything needed to use it

A PYTORCH TRAINING LOOP YOU CAN TRUST

Save the state dict with everything needed to use it

```
torch.save(model.state_dict(), run_dir / "weights.pt")
joblib.dump(scaler, run_dir / "scaler.joblib")
(run_dir / "run.json").write_text(json.dumps(metadata, indent=2))

restored = build_model(config["width"], config["dropout"])
state = torch.load(run_dir / "weights.pt",
                   map_location="cpu", weights_only=True)
restored.load_state_dict(state)
restored.eval()
check = torch.sigmoid(restored(X_val_scaled)).detach().numpy().ravel()
np.testing.assert_allclose(check, val_prob, rtol=1e-6, atol=1e-6)
```

The reload check proves the saved files reproduce the predictions you validated.

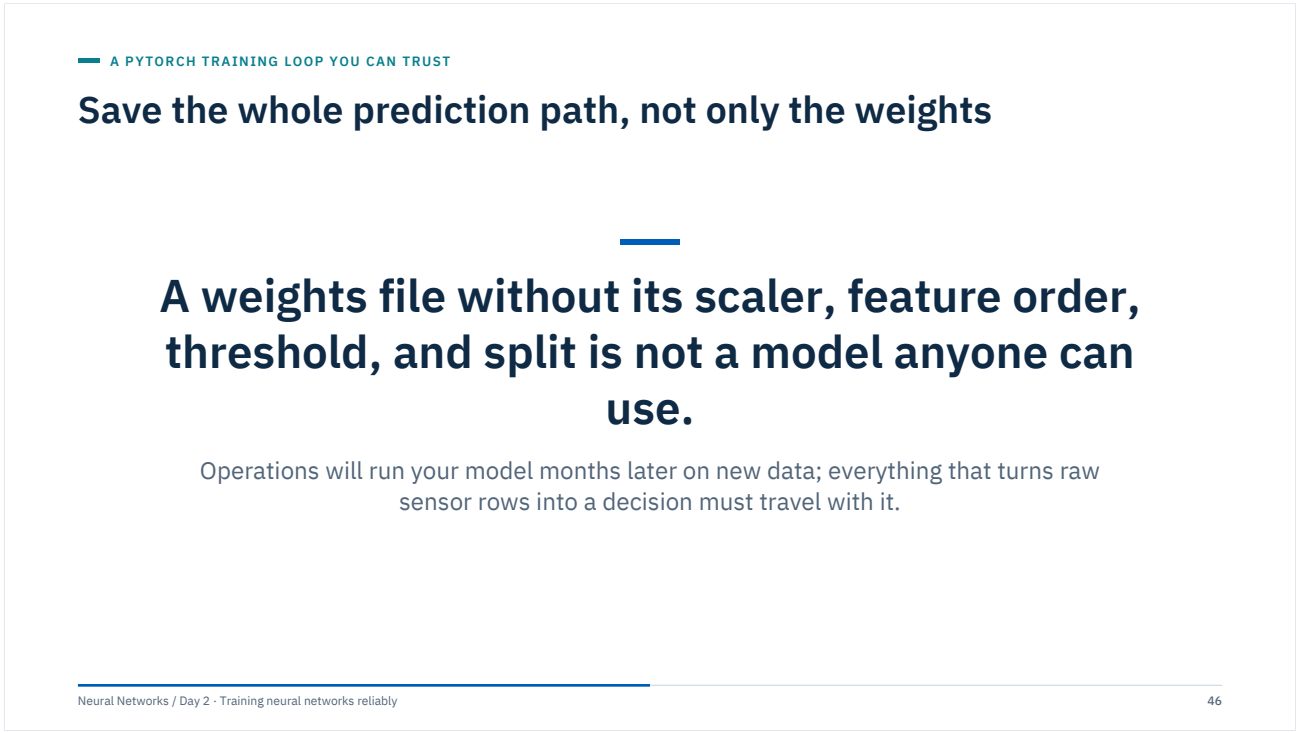
Neural Networks / Day 2 · Training neural networks reliably 45

Save the state dictionary, not the whole pickled model object: a state dict is a plain mapping from names to tensors that loads with weights_only=True, which avoids executing arbitrary pickled code and survives refactoring of your class. The model is rebuilt from the same configuration and then receives the weights.

The scaler and metadata are saved next to the weights: the configuration, selected epoch, feature order, label mapping, threshold, split indices and library versions. map_location="cpu" lets a model trained on a GPU load on a laptop.

The last two lines are the reload check from Lab 1: predictions from the restored model and saved scaler must match the validated predictions to within floating-point tolerance. If they do not, something in the saved path differs, for example a scaler refitted on other rows or a model left in training mode. [2 min] @source https://pytorch.org/tutorials/beginner/saving_loading_models.html

Save the whole prediction path, not only the weights

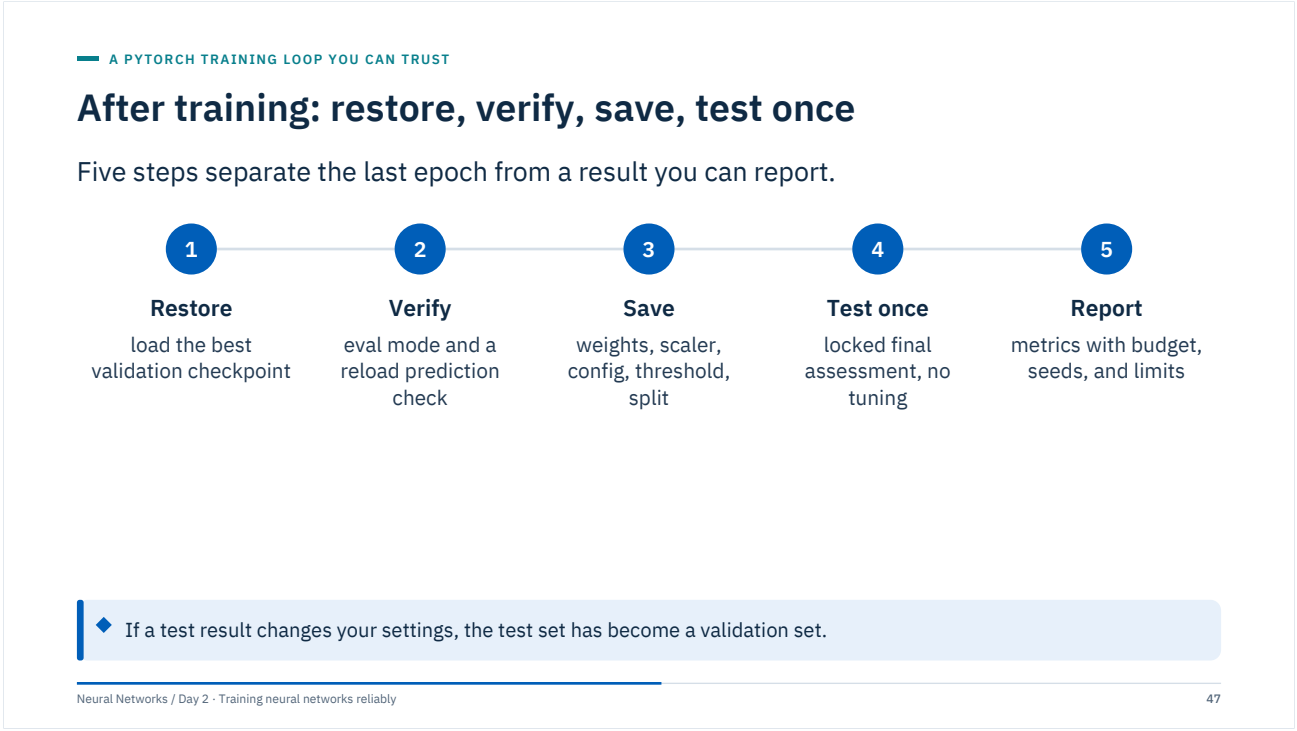


Pause here for emphasis. In energy operations a model is rarely used by the person who trained it. A pump-surveillance model trained in March is run in September by someone else, on data exported from a different historian, sometimes with columns in a different order.

The weights encode only the last step of the prediction path. The scaler encodes what "normal" means for each input; the feature order says which column is which; the threshold turns a probability into an alarm; the split indices say which data the reported score came from. Lose any of these and the model either fails or, worse, runs and gives wrong answers.

Ask: in your organization, where would these artifacts live, and who would be able to find them in a year? [1 min]

After training: restore, verify, save, test once



These are the after-training steps, in order. Restore the checkpoint selected on validation data, because the model in memory is usually a few epochs past it. Verify: switch to eval mode, and check that saving and reloading reproduces the validated predictions. Save everything the prediction path needs.

Only then run the final test, once, on data that has not influenced any choice. In Lab 1 this is the RUN_FINAL flag, which stays False until the whole procedure is fixed. Report the metrics together with the training budget, the seeds used, the size of the evaluation set and what the data cannot establish: for Sonar, there are no acquisition groups, so the result says nothing about new sensors or sites.

The callout is the key rule: the moment a test result feeds back into a design decision, it is no longer an unbiased estimate. [2 min]

Mixed precision is a GPU speed option, not a default

A PYTORCH TRAINING LOOP YOU CAN TRUST

Mixed precision is a GPU speed option, not a default

```
use_amp = device.type == "cuda"
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(
            device_type=device.type,
            dtype=torch.bfloat16,
            enabled=use_amp):
        loss = criterion(model(xb), yb)
    loss.backward()
    optimizer.step()
```

- Autocast runs matrix multiplies in 16-bit where safe
- bfloat16 needs no loss scaling; float16 needs a GradScaler
- Compare losses and metrics with a float32 run
- Little benefit for small tabular models

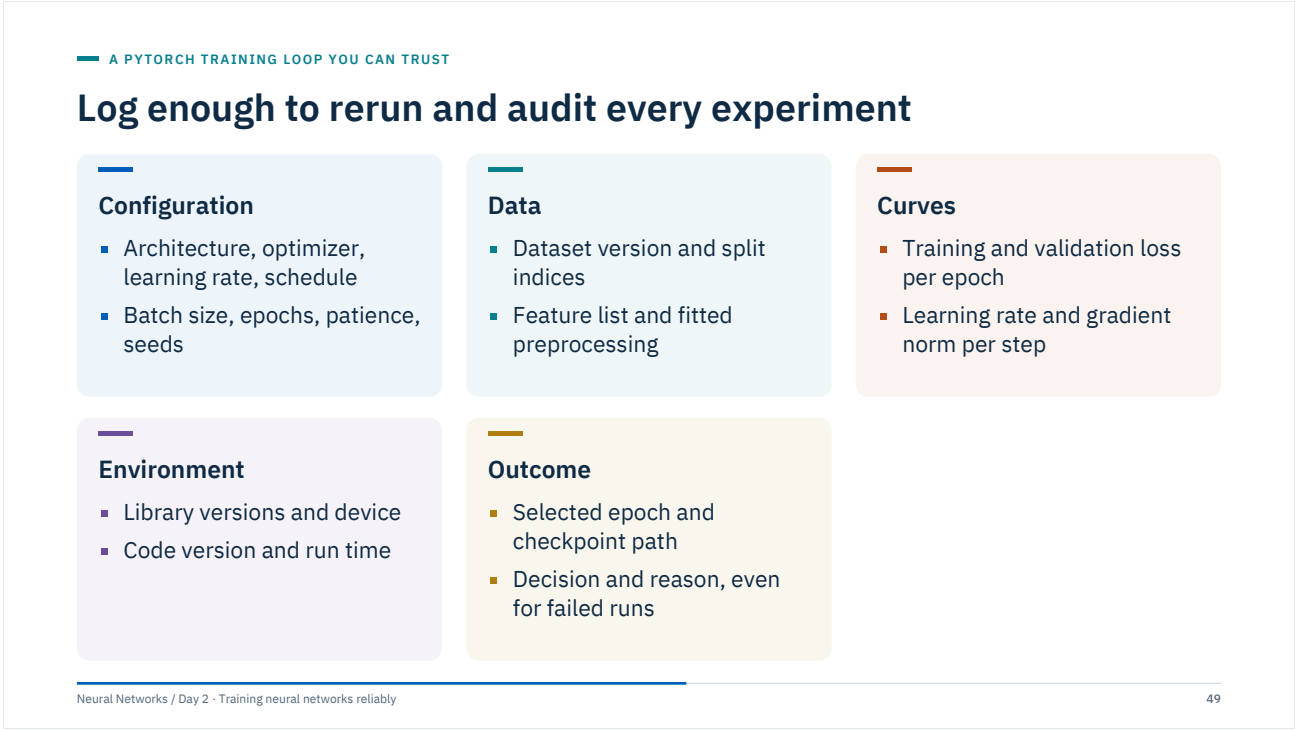
Neural Networks / Day 2 · Training neural networks reliably 48

Mixed precision runs selected operations, mainly matrix multiplications and convolutions, in 16-bit floating point while keeping sensitive operations and the master weights in 32-bit. On modern GPUs this can speed up training and reduce memory for large models.

Two 16-bit formats exist. bfloat16 keeps the exponent range of float32, so gradients rarely underflow and no loss scaling is needed; it requires hardware support. float16 has a smaller range, so small gradients can underflow to zero; it is used with a gradient scaler (torch.amp.GradScaler("cuda") in recent PyTorch releases, torch.cuda.amp.GradScaler() in older ones) that scales the loss before backward and unscales before the step.

The backward call is outside the autocast block, as recommended. Treat mixed precision as an optional performance extension: verify that the loss curve and validation metrics match a float32 run before trusting it. For the small models in this course the benefit is negligible. [2 min] @source https://docs.pytorch.org/docs/stable/amp.html

Log enough to rerun and audit every experiment



Experiment tracking is simply the discipline of writing down, automatically, everything needed to rerun and audit a training run. Tools such as MLflow, Weights & Biases or TensorBoard help, but a JSON file and a CSV of curves per run, as in Lab 1, already cover most of it.

Walk through the cards. Configuration and data define the experiment. Curves are the evidence you will look at when something goes wrong; logging the learning rate matters as soon as you use a schedule. Environment explains why a rerun on another machine differs slightly. The outcome card is the one most often skipped: record what you decided and why, including runs that failed, so that nobody repeats them.

Ask: if a colleague asked you today to reproduce your best model from six months ago, which of these would you be missing? [2 min]

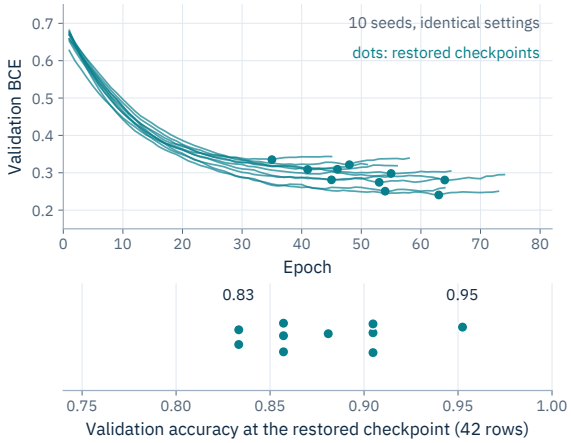
Reproducibility includes the experiment definition

A PYTORCH TRAINING LOOP YOU CAN TRUST

Reproducibility includes the experiment definition

- Record split indices, seeds, dataset version, and feature schema
- Save preprocessing, configuration, weights, and threshold
- Log library versions and hardware
- Repeat promising settings over several seeds

◆ Ten seeds, one configuration: validation accuracy from 0.83 to 0.95 on 42 rows.



Computed from the course file sonar.all-data.csv with the Lab 1 split and training loop; torch seeds 0–9.

Neural Networks / Day 2 · Training neural networks reliably 50

The figure uses the real Lab 1 data and code with ten different torch seeds and nothing else changed. The restored checkpoints come from epochs 35 to 64, and validation accuracy ranges from 0.833 (35 of 42 rows) to 0.952 (40 of 42). With 42 validation rows, one row is worth 2.4 percentage points, so a difference of one or two rows between two settings is well within seed-to-seed variation.

That is why the reproducibility checklist has two halves: record everything needed to rerun one run exactly, and run promising settings over several seeds before claiming one is better.

Identical seeds do not guarantee bitwise agreement across every device and library version, and GPU kernels can be non-deterministic. PyTorch offers torch.use_deterministic_algorithms(True) at some speed cost. Measure actual training and inference time rather than assuming a GPU will help a tiny tabular dataset. [3 min] @source https://docs.pytorch.org/docs/stable/notes/randomness.html

The same contract in Keras

A PYTORCH TRAINING LOOP YOU CAN TRUST

The same contract in Keras

```
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(n_features,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2), keras.layers.Dense(1)
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.BinaryCrossentropy(from_logits=True)
)
```

Same architecture, same raw-logit output, same loss; `fit()` hides the loop you just wrote.

Neural Networks / Day 2 · Training neural networks reliably 51

This Keras 3 example requires a configured backend, such as TensorFlow, JAX or PyTorch; the Lab 1 alternative notebook uses TensorFlow, which must be installed on the machine. The contract is the same as the PyTorch model: 60 inputs, one hidden layer of 32 ReLU units, dropout, and a final Dense(1) that outputs a raw logit. The loss uses from_logits=True, the equivalent of BCEWithLogitsLoss.

Choose a sigmoid at inference, or use from_logits=False only together with a sigmoid output layer. Mixing a sigmoid output with from_logits=True reproduces exactly the double-sigmoid bug from the one-batch test.

The Keras notebook uses a two-hidden-layer model with a sigmoid output; ask learners to spot which loss setting then applies. Note that Keras's Dropout rate is also the drop probability. [2 min] @source https://keras.io/api/

Explicit validation is easier to audit

A PYTORCH TRAINING LOOP YOU CAN TRUST

Explicit validation is easier to audit

```
stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10,
    restore_best_weights=True
)
model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=100, batch_size=32, callbacks=[stop]
)
```

Prefer `validation_data` over `validation_split`: you choose the rows, and the scaler never sees them.

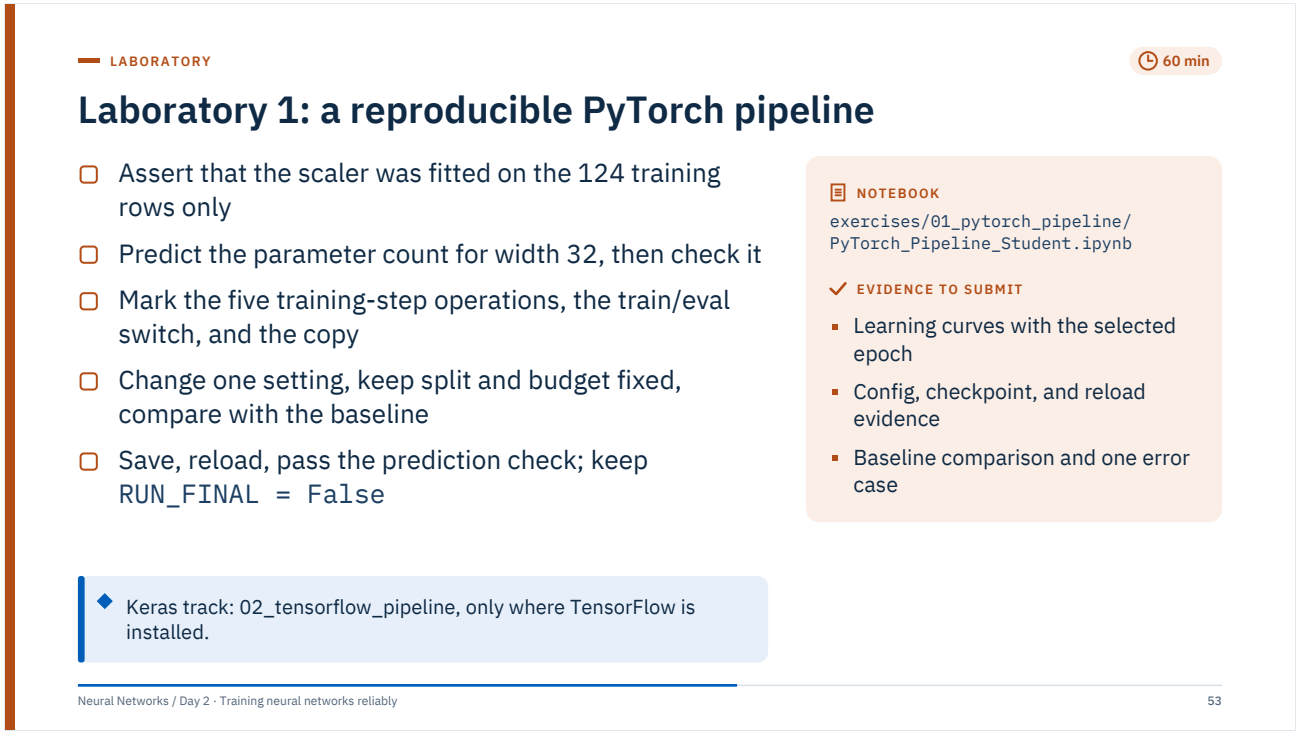
Neural Networks / Day 2 · Training neural networks reliably 52

Fit the scaler before this call using only the training partition. A validation_split applied after scaling the entire input array has already allowed validation statistics into preprocessing; Keras's validation_split also takes the last fraction of the array without shuffling, which is dangerous if rows are ordered by well or time. Grouped and time splits should be explicit rather than delegated to a generic random split.

restore_best_weights=True is Keras's version of the copied checkpoint: without it, the model left after fit is the one from the final epoch, not the best one.

The Keras notebook in Lab 1's alternative track follows this pattern: training, validation and test partitions made before scaling (the same rows as the PyTorch notebook), validation_data with restore_best_weights, architectures compared on validation, and one final test of the chosen model. Ask learners to find where each of these happens. [2 min] @source https://keras.io/api/callbacks/early_stopping/

Laboratory 1: a reproducible PyTorch pipeline



Timing: 60 minutes before lunch, debrief at 13:30. Suggested pacing: 10 minutes for sections 1–2, 15 for section 3 and the curves, 20 for the controlled experiment, 15 for saving and the reload check. [60 min]

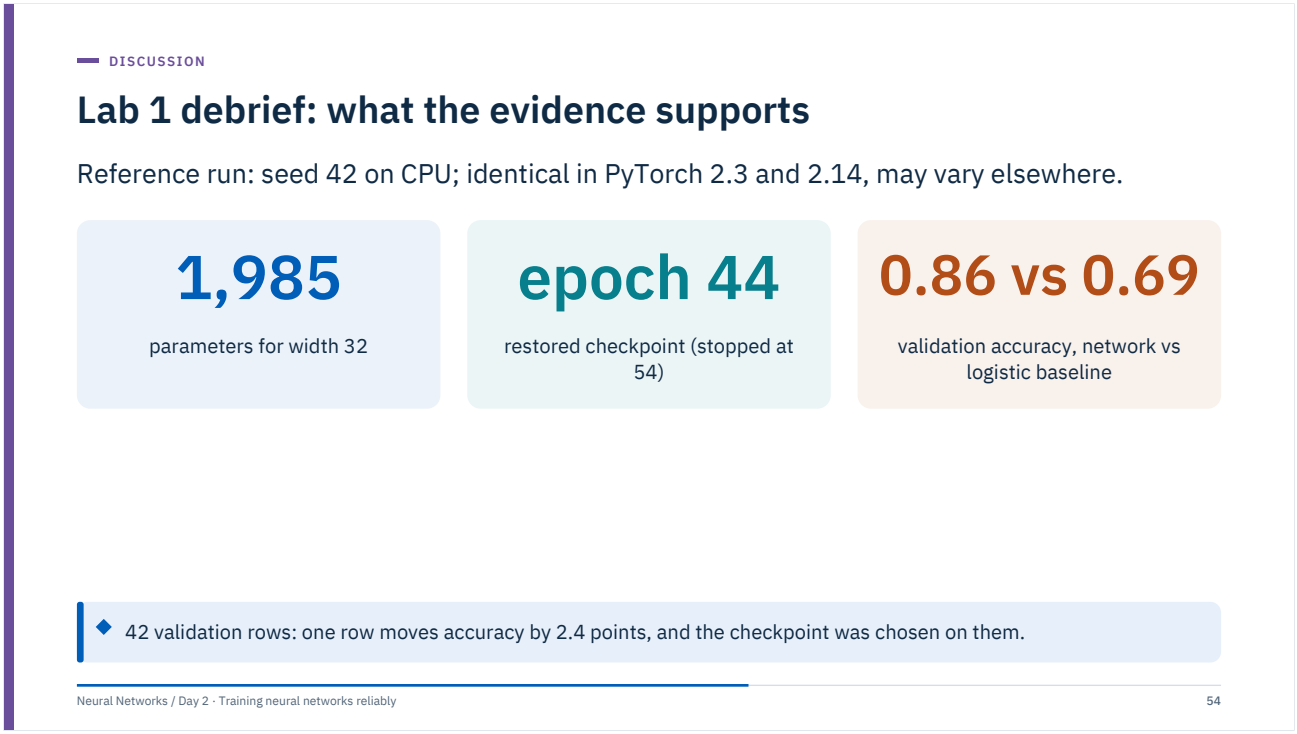
Setup: the notebook runs on the CPU with one thread and needs numpy, pandas, scikit-learn, matplotlib, joblib and PyTorch; no downloads. The split is 124 training, 42 validation and 42 reserved final rows, made before any scaling.

Worked answers. Section 1: np.testing.assert_allclose(scaler.mean_, X_train.mean(axis=0), rtol=1e-5); the scaler may transform validation rows but must not learn from them. Section 2: 60 × 32 + 32 + 32 × 1 + 1 = 1,985 parameters; output shape [batch, 1]; no sigmoid because BCEWithLogitsLoss applies it. Section 3: zero_grad, forward, loss, backward, step; model.train() at the start of each epoch and model.eval() before validation; copy.deepcopy(model.state_dict()); a shallow reference would change with later training. Section 5: one change with a stated hypothesis, for example dropout 0.2 to 0.4 or width 32 to 64; same split seed, budget and threshold. Section 6: the reload check must pass.

Common errors: refitting the scaler on all rows; comparing a change on a different split; reading the final block before choosing; claiming improvement from a one-row difference.

Alternative track: learners with TensorFlow 2.16 or later (which brings Keras 3) can work through 02_tensorflow_pipeline. It maps Rock = 0 and Mine = 1, uses the same 124 / 42 / 42 partitions as this notebook, passes explicit validation_data, compares architectures on the validation rows and tests the chosen one once. TensorFlow is not in requirements-labs.txt, so run the notebook once on a classroom machine before offering this track.

Lab 1 debrief: what the evidence supports



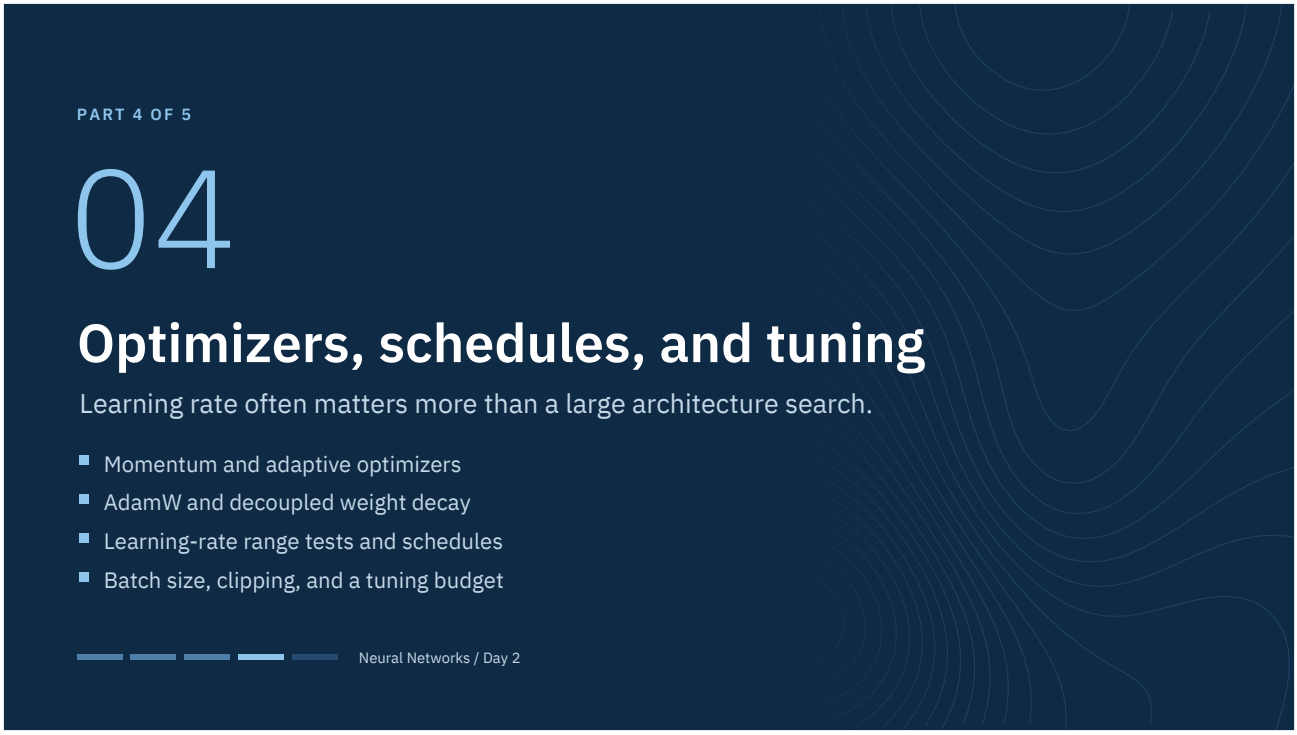
Start the afternoon with this debrief. [10 min]

Reference numbers from running the notebook exactly as supplied (reproduced unchanged with PyTorch 2.14): 124 / 42 / 42 rows for training, validation and final; 1,985 parameters; early stopping selected epoch 44 (validation loss 0.307) and stopped at epoch 54 after 10 stale epochs; validation accuracy 0.857 (36 of 42) against 0.690 (29 of 42) for the logistic-regression baseline trained on the same scaled rows.

Ask two or three learners to report their controlled change and what happened. Then make three points. First, the checkpoint and the change were both chosen on these 42 rows, so the validation accuracy is optimistic; that is what the reserved final set is for. Second, the seed study earlier showed 0.83 to 0.95 across seeds for the unchanged configuration, so a single-run difference of one or two rows is not evidence. Third, the Sonar data have no acquisition groups, so nothing here says how the model behaves on a new sonar system or site.

If someone ran the final block, discuss honestly what that means for their later choices.

Optimizers, schedules, and tuning



Part 4 runs from about 13:40 to 14:35. Recall one gradient step from Day 1: $w \leftarrow w - \eta \times \text{gradient}$. Optimizers transform gradient information into updates; they differ in how they use past gradients and how they scale steps per parameter. Their default settings are starting points that require validation.

The second half of this part is about spending a limited training budget well: finding a usable learning rate quickly, choosing a schedule, understanding what batch size changes, and planning a small search that answers a real question. [1 min]

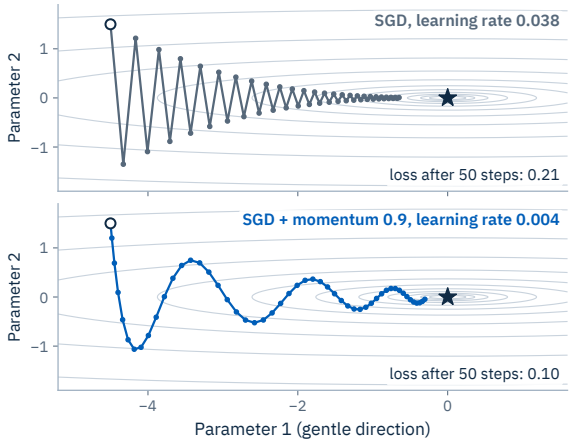
SGD with momentum accumulates direction

OPTIMIZERS, SCHEDULES, AND TUNING

SGD with momentum accumulates direction

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

- *v* is a running sum of past gradients, weighted by β
- Zig-zags across the valley cancel; steady progress along it adds up
- The learning rate still sets the overall step scale



Computed with torch.optim on L = (w1² + 50 w2²)/2, 50 steps from (−4.5, 1.5); learning rates chosen for illustration.

Neural Networks / Day 2 · Training neural networks reliably 56

Symbols: g_t is the gradient at step t, v_t the velocity, β the momentum coefficient (0.9 here), η the learning rate, θ the parameters. This is the convention PyTorch's SGD uses; some texts put (1 − β) in front of the gradient, which rescales the effective learning rate. Avoid mixing formulas and implementation parameter meanings.

The bowl is 50 times steeper in the vertical direction than the horizontal one, a toy version of the ill-conditioning common in neural-network losses. Plain SGD cannot use a learning rate above 2/50 = 0.04 without diverging in the steep direction; at 0.038 it zig-zags and after 50 steps has covered only 85% of the distance along the gentle direction (loss 0.21). With momentum, the up-and-down components of successive gradients cancel in v while the consistent horizontal component accumulates, so a learning rate ten times smaller travels further (loss 0.10). Momentum does not guarantee escape from every poor region; it changes the dynamics. [3 min]

Adaptive optimizers track gradient scale

OPTIMIZERS, SCHEDULES, AND TUNING

Adaptive optimizers track gradient scale

Optimizer	Running information	Practical consideration
RMSprop	Squared gradients	Adapts step sizes per parameter
Adam	Gradients and squared gradients	Bias-corrected moment estimates
AdamW	Adam-style moments plus decoupled decay	Separates decay from adaptive gradient scaling

Each still has a learning rate that must be tuned; defaults are starting points.

Adaptive optimizers keep a running average of squared gradients for every parameter and divide the step by its square root. Parameters with consistently large gradients take smaller steps; parameters with small gradients take relatively larger ones. RMSprop does only this. Adam adds momentum on top, using a running average of the gradients themselves, and corrects both averages for starting at zero. AdamW is Adam with weight decay applied separately from the gradient; two slides ahead explain why that matters.

Adam still has a learning rate, and it can generalize differently from SGD with momentum; in some domains well-tuned SGD is still preferred. Match the training budget and validation protocol when comparing optimizers: an optimizer that looks better at its default settings may simply have been tuned more. [2 min] @source https://arxiv.org/abs/1412.6980

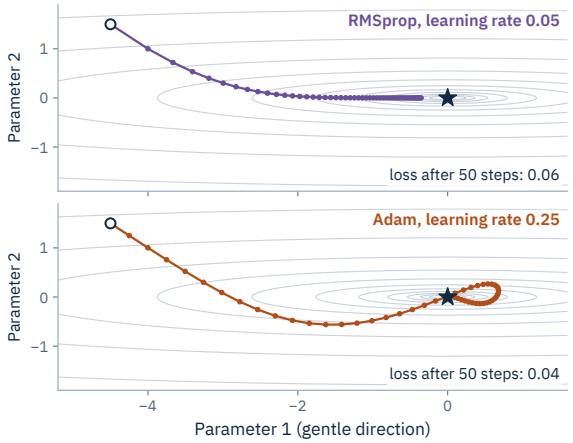
Adam's update uses two moment estimates

OPTIMIZERS, SCHEDULES, AND TUNING

Adam's update uses two moment estimates

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- *m* tracks direction and *v* tracks squared size, per parameter
- Hats mark the correction for averages that start at zero
- Early steps are about η per parameter, whatever the gradient size
- ε keeps the denominator away from zero



Computed with torch.optim (default betas and alpha) on the same bowl; learning rates chosen for illustration.

Neural Networks / Day 2 · Training neural networks reliably 58

Optional equations for the notes: m_t = β1 m_{t−1} + (1 − β1) g_t and v_t = β2 v_{t−1} + (1 − β2) g_t², with defaults β1 = 0.9 and β2 = 0.999; the hats divide by (1 − β^t) to correct for starting at zero. Day 2's core requirement is recognizing why the update differs from raw SGD, not memorizing every default constant.

Because the step divides by the square root of v̂, Adam's first steps are close to η in each coordinate regardless of how steep that coordinate is; RMSprop has no bias correction, so its first step is about 10η per coordinate (0.5 here). Either way, on the bowl both optimizers head diagonally at first instead of plunging down the steep side. RMSprop (top) then slides along the valley floor; Adam (bottom) carries momentum, overshoots the minimum slightly and curls back. After 50 steps both are closer to the minimum than SGD.

Do not over-read the ranking: on a different problem or with other learning rates the order changes. The shapes, not the winner, are the lesson. [3 min]

A fair comparison tunes each optimizer's learning rate

OPTIMIZERS, SCHEDULES, AND TUNING

A fair comparison tunes each optimizer's learning rate

An optimizer run at someone else's learning rate is not a fair competitor.

Give every candidate the same budget, the same split, several seeds, and its own small learning-rate search.

Neural Networks / Day 2 · Training neural networks reliably 59

A short pause for a principle that is violated constantly in practice and in published comparisons. Each optimizer has a different useful range of learning rates: the bowl illustrations used 0.038 for SGD, 0.004 for momentum, 0.05 for RMSprop and 0.25 for Adam, a spread of about 60 times. Comparing all four at one shared learning rate would mostly measure which one happens to like that rate.

A fair comparison fixes the budget (number of updates or wall-clock time), the data split and the evaluation metric, runs a few seeds for each candidate, and gives each candidate its own small learning-rate search. Only then is the comparison about the optimizer.

Ask: if a colleague reports that Adam beat SGD by 2% on a 40-row validation set with one seed each, what would you ask for? [1 min]

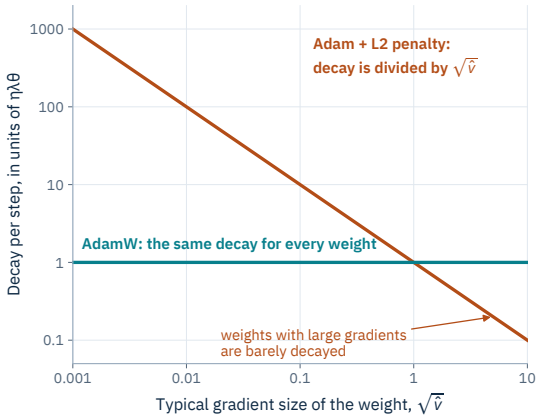
AdamW decouples weight decay from the adaptive step

OPTIMIZERS, SCHEDULES, AND TUNING

AdamW decouples weight decay from the adaptive step

$$\text{Adam} + \text{L2}: \; g_t \leftarrow g_t + \lambda\theta_t \text{ before scaling}$$
$$\text{AdamW}: \; \theta_{t+1} = \theta_t - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta_t\right)$$

- With L2 inside Adam, the adaptive denominator divides the penalty too
- Weights with large gradients are then barely regularized
- `Adam(weight_decay=...)` is the L2 form; `AdamW` is decoupled



Analytical approximation, valid while the decay term is small relative to the gradient.

Neural Networks / Day 2 · Training neural networks reliably 60

Symbols: λ is the weight-decay coefficient, θ the parameters, η the learning rate. With plain SGD, adding an L2 penalty to the loss and shrinking the weights directly are the same thing. With adaptive optimizers they are not. If the penalty gradient λθ is added to g before Adam's scaling, it is divided by √v̂ like everything else: weights whose gradients are typically large (large v̂) receive almost no decay, and weights with small gradients receive a lot.

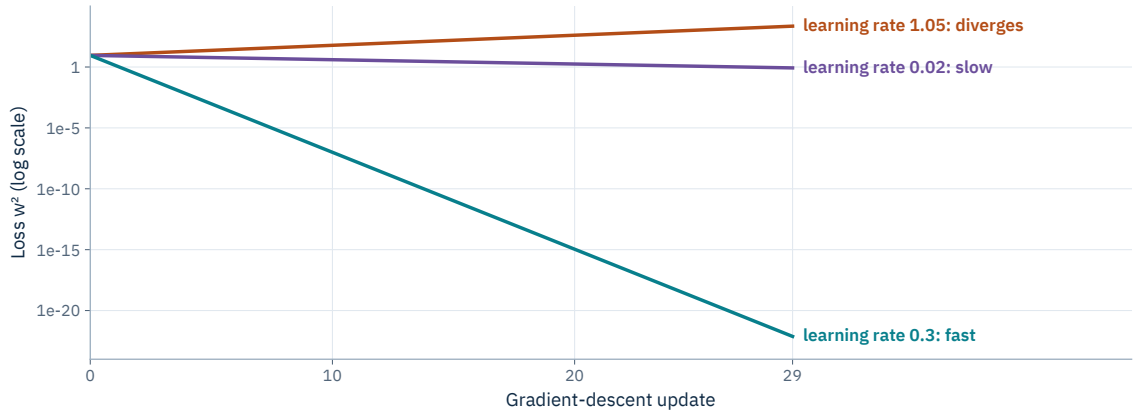
AdamW (Loshchilov and Hutter, 2019) applies the decay outside the adaptive scaling, so every weight shrinks by the same fraction ηλ per step. In PyTorch, torch.optim.Adam's weight_decay argument implements the L2 form; torch.optim.AdamW implements the decoupled form, with a default weight_decay of 0.01.

This is also why AdamW's weight_decay values are not interchangeable with Adam's; retune when you switch. The MNIST notebook in Lab 2 uses Adam with weight_decay, the L2 form. [3 min] @source https://arxiv.org/abs/1711.05101 ; https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html

Step size changes convergence

OPTIMIZERS, SCHEDULES, AND TUNING

Step size changes convergence



Computed: exact gradient descent on the loss w² from w = 3 with three learning rates. Large steps can diverge even in a simple problem.

Neural Networks / Day 2 · Training neural networks reliably 61

Read the loss axis (log scale) and compare progress per step. The loss is w^2 starting at $w = 3$, so each gradient step multiplies w by $(1 - 2\eta)$. With $\eta = 0.02$ the factor is 0.96: steady but slow progress. With $\eta = 0.3$ the factor is 0.4: fast convergence. With $\eta = 1.05$ the factor is -1.1: every step overshoots further and the loss grows without bound.

Real neural objectives are more complex, but this isolates the effect of step size. Two practical lessons: a very low learning rate can look like a broken training loop over a short run, and a learning rate slightly too high can look like noisy data. The next slide shows how to find the usable range directly instead of guessing. [2 min]

A learning-rate range test finds the usable window

OPTIMIZERS, SCHEDULES, AND TUNING

A learning-rate range test finds the usable window

- Raise the rate every mini-batch, starting from 1e-6
- Stop when the smoothed loss climbs well above its start
- A common heuristic: start about 10× below the loss minimum
- Rerun after changing batch size, model, or optimizer



Computed: synthetic ESP table (4,000 rows, seed 3), MLP 12-64-64-1, AdamW, batch 32, torch seed 0; grey = raw batch loss, blue = moving average.

Neural Networks / Day 2 · Training neural networks reliably 62

The range test (Smith, 2017) is a single short run in which the learning rate grows geometrically every mini-batch. At first nothing moves: the rate is too small. Then the loss starts to fall; the steepest part of the curve marks rates that learn quickly. Eventually the steps are too large and the loss rises and blows up.

In this run the smoothed loss is lowest near a learning rate of 0.024. The heuristic of starting about ten times lower suggests 0.002, and the shaded band marks a factor of three either side; the AdamW runs earlier in the day used 0.001, inside that band.

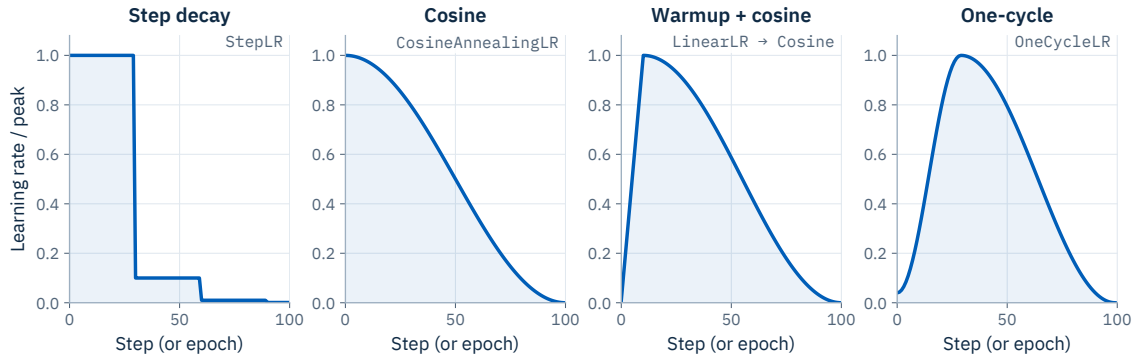
Caveats: the curve depends on the model, batch size, optimizer and even the initialization, so rerun it when any of these change. It is a quick way to find the right order of magnitude, not a precise optimum; confirm with a few short training runs on validation data. [3 min] @source https://arxiv.org/abs/1506.01186

Schedules are part of the experiment

OPTIMIZERS, SCHEDULES, AND TUNING

Schedules are part of the experiment

Step, cosine, warmup, and one-cycle schedules shape the same peak rate over a fixed budget.



Computed by stepping the named torch.optim.lr_scheduler classes for 100 steps; peak rate scaled to 1.

Neural Networks / Day 2 · Training neural networks reliably 63

A schedule changes the learning rate during training: typically larger steps early to make progress and smaller steps late to settle into a minimum. Step decay (StepLR) cuts the rate by a factor at fixed epochs, here by 10 every 30. Cosine annealing reduces it smoothly to near zero over the chosen budget. Warmup increases the rate from a small value over the first steps, here 10, which protects the first updates when the optimizer's statistics are still unreliable; it is standard for transformers and large batches. One-cycle (Smith and Topin) rises to a peak at 30% of the budget and anneals to a very small value; by default PyTorch's OneCycleLR also cycles the momentum.

Plateau schedules (ReduceLROnPlateau) react to a validation metric instead of a clock. Do not tune schedules on the final test. The optimizer, schedule, batch size and number of updates jointly define the training experiment; change one and the others may need adjusting. [3 min] @source https://docs.pytorch.org/docs/stable/optim.html ; https://arxiv.org/abs/1608.03983 ; https://arxiv.org/abs/1708.07120

Schedulers step on a declared clock

OPTIMIZERS, SCHEDULES, AND TUNING

Schedulers step on a declared clock

```python
from torch.optim import lr_scheduler as S

steps = epochs * len(train_loader)
warm = steps // 10
warmup = S.LinearLR(optimizer, start_factor=0.01, total_iters=warm)
cosine = S.CosineAnnealingLR(optimizer, T_max=steps - warm)
sched = S.SequentialLR(optimizer, [warmup, cosine], milestones=[warm])

for epoch in range(epochs):
    for xb, yb in train_loader:
        ...                          # zero_grad, forward, backward
        optimizer.step()
        sched.step()                 # once per batch: the clock
```

`ReduceLROnPlateau` is the exception: call `sched.step(val_loss)` once per epoch.

The most common scheduler bug is stepping on the wrong clock. Here the schedule is defined in optimizer steps (epochs × batches per epoch), so sched.step() is called once per batch, after optimizer.step(). If you define T_max in epochs but step per batch, the cosine finishes in the first few epochs; the reverse leaves the rate almost unchanged.

Write the clock in the configuration ("scheduler stepped per batch, 400 steps total") and log the learning rate every step so the curve can be checked afterwards.

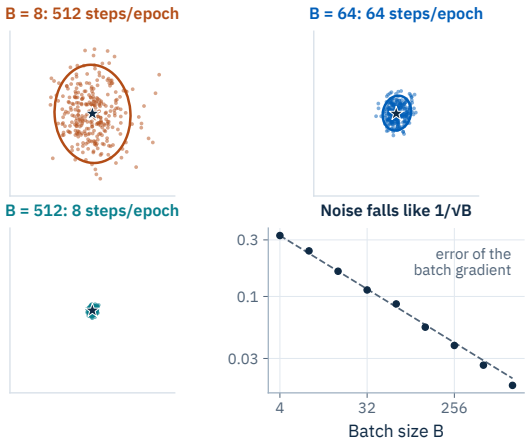
SequentialLR chains the linear warmup and the cosine decay at the milestone. Some PyTorch versions print a deprecation warning about an epoch argument from SequentialLR's internal calls; it is harmless. ReduceLROnPlateau is different: it needs the monitored metric, so it is stepped once per epoch after validation. [2 min]

Batch size changes both compute and learning

OPTIMIZERS, SCHEDULES, AND TUNING

Batch size changes both compute and learning

- Larger batches may improve hardware throughput
- Smaller batches add gradient noise and more updates per epoch
- Memory depends on activations as well as parameters
- Compare update counts or compute budgets, not epochs alone



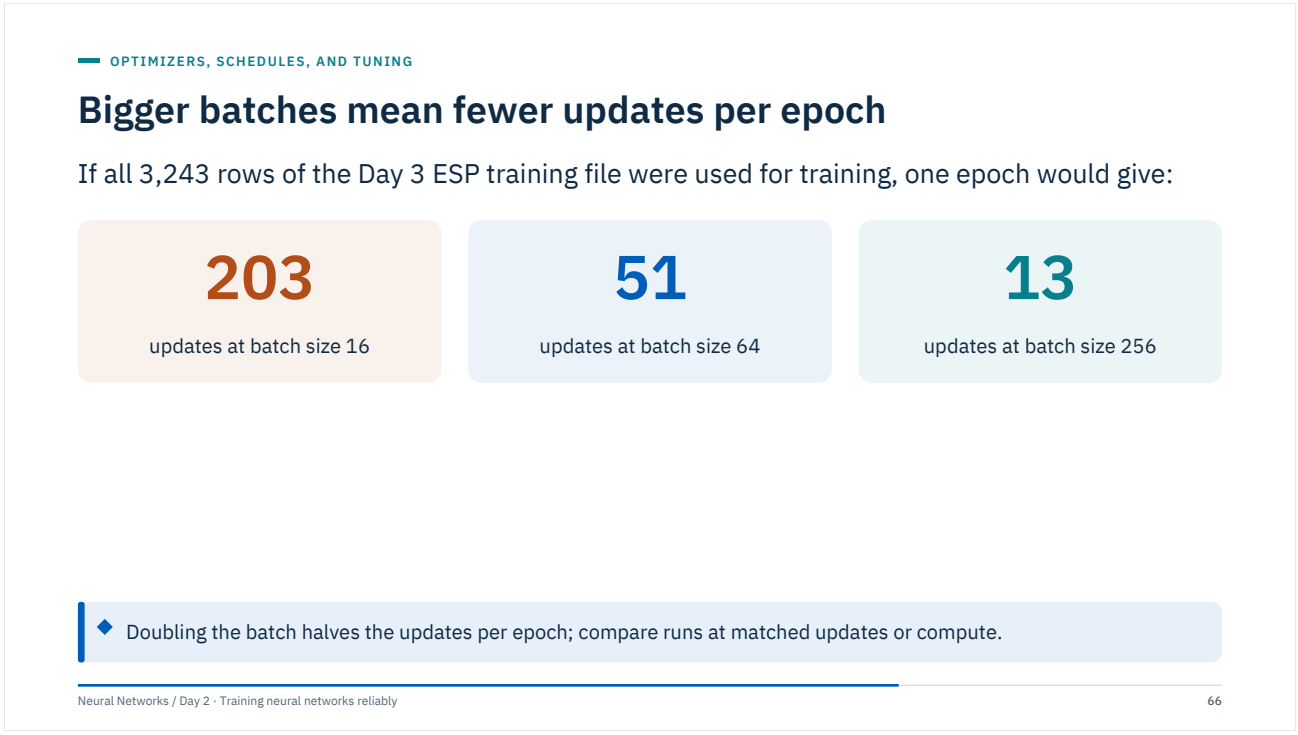
Computed: logistic-regression gradients at one fixed point, 4,096 synthetic rows (numpy seed 0), 300 random mini-batches per size.

Neural Networks / Day 2 · Training neural networks reliably 65

Each dot is the gradient computed from one random mini-batch at the same parameter values; the star is the full-batch gradient. With batches of 8 the estimates scatter widely; with 512 they sit almost on the star. The bottom-right panel shows that the error of the batch gradient falls like one over the square root of the batch size: 64 times more rows per batch reduces the noise about 8 times. At B = 1,024 the point sits slightly below the line because the batches are a quarter of the whole dataset.

That noise is not all bad: it lets small-batch training explore and may help generalization, while large batches give cleaner steps and better hardware use. An epoch is one pass through the training data, so a fixed number of epochs does not imply the same number of optimizer steps after a batch-size change. Gradient accumulation can simulate some aspects of larger batches, but batch normalization and stochastic operations may behave differently. [3 min] @source https://arxiv.org/abs/1706.02677

Bigger batches mean fewer updates per epoch



The arithmetic is ceiling(N / B) with the last partial batch kept: 3,243 / 16 = 202.7, so 203 updates; 3,243 / 64 = 50.7, so 51; 3,243 / 256 = 12.7, so 13. A run of 50 epochs therefore makes 10,150 updates at batch size 16 but only 650 at batch size 256.

If you increase the batch size and keep the epochs fixed, you have also cut the number of updates, and the learning rate that worked before may now be too small. A common heuristic for large-batch training is to scale the learning rate with the batch size and add warmup (Goyal et al., 2017), but for small tabular models it is simpler to retune.

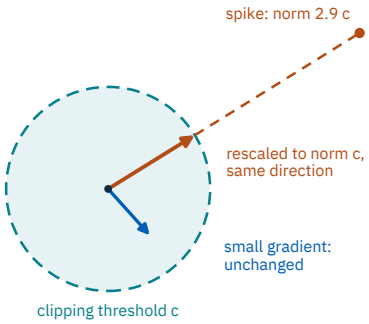
In Day 3 the ESP data are split by pump before any training, so the real training partition will be smaller than 3,243 rows; the principle is the same. [2 min]

Gradient clipping caps rare, oversized steps

OPTIMIZERS, SCHEDULES, AND TUNING

Gradient clipping caps rare, oversized steps

- Rescales the whole gradient when its global norm exceeds c
- Direction is kept; only the step length is capped
- Call after `backward()`, before `optimizer.step()`
- Log how often it triggers; constant clipping hides a bad rate



$$g \leftarrow g \cdot \min(1,\ c / \|g\|)$$

one global norm over all parameters

Schematic of gradient-norm clipping in two dimensions.

Neural Networks / Day 2 · Training neural networks reliably 67

Clipping by norm computes one global norm over all parameters' gradients. If it exceeds the threshold c, every gradient is multiplied by c divided by that norm, so the update keeps its direction but its length is capped. Small gradients are unchanged. In PyTorch: torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=c), which also returns the norm before clipping, useful for logging.

Clipping is standard for recurrent networks (Pascanu et al., 2013, discussed in Day 5) and common in transformers, where rare batches produce very large gradients. It is a safety device against spikes, not a cure for a learning rate that is too high: if clipping triggers on most steps, the effective update is always c times the learning rate and the real problem is hidden. It also does nothing for vanishing gradients. [2 min] @source https://arxiv.org/abs/1211.5063

Plan a fair optimizer comparison

PRACTICE

6 min Pairs

Plan a fair optimizer comparison

You will compare SGD with momentum against AdamW on the ESP table (3,243 training rows).

1. With batch size 32 for 50 epochs, how many optimizer updates is that?
2. A colleague switches to batch size 64 but keeps 50 epochs. What changed besides the batch?
3. What must be tuned separately for each optimizer before comparing them?

Neural Networks / Day 2 · Training neural networks reliably 68

Four minutes in pairs, two to debrief. [6 min]

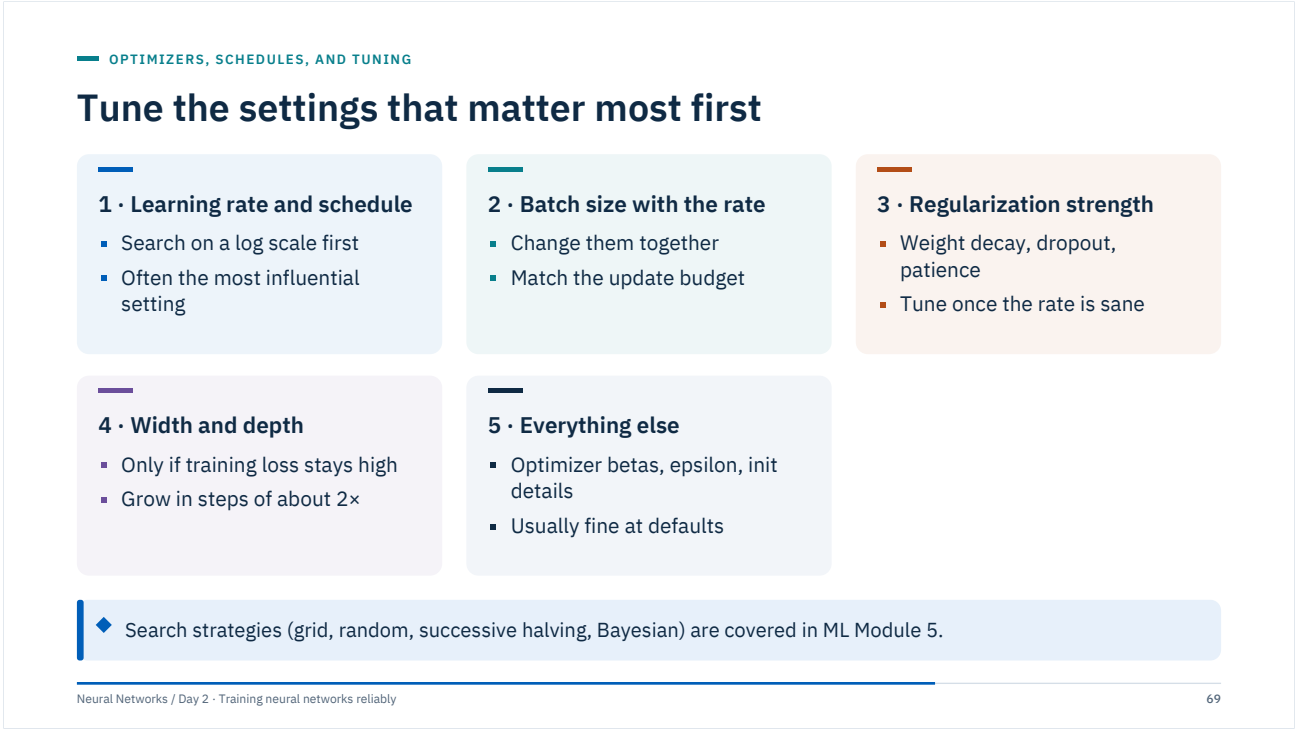
1. ceiling(3,243 / 32) = ceiling(101.3) = 102 updates per epoch; × 50 epochs = 5,100 updates.

2. At batch size 64 there are ceiling(50.7) = 51 updates per epoch, so 2,550 updates in 50 epochs: half as many optimizer steps. Each gradient is also less noisy (about 1/√2 of the noise), the best learning rate may shift, and any schedule defined in steps now ends at a different point. Wall-clock time per epoch may fall on a GPU. A fair comparison matches the number of updates or the compute, not the number of epochs.

3. The learning rate (with a short range test or log-scale search) and, if used, the schedule, for each optimizer separately. Keep the split, the budget, the evaluation metric and the seeds fixed, and repeat each final candidate over several seeds. SGD usually also needs its momentum checked; AdamW's weight decay should be tuned because it is not equivalent to an L2 penalty.

Common error: comparing at the default learning rate of one library for both optimizers.

Tune the settings that matter most first



This ordering is practical guidance for small and medium models, not a law. Goodfellow, Bengio and Courville (2016, section 11.4) single out the learning rate as the hyperparameter to tune if you can tune only one, and most practical guides, such as the Deep Learning Tuning Playbook, start there too.

Batch size and learning rate interact, so tune them together or fix the batch size for hardware reasons and tune the rate. Regularization strength matters once the model can fit the training data; tuning dropout while the learning rate is wrong wastes runs. Width and depth come later than people expect: on tabular energy data, a wider or deeper network rarely fixes what a better learning rate or better features would. Optimizer internals are rarely worth tuning for this scale of problem.

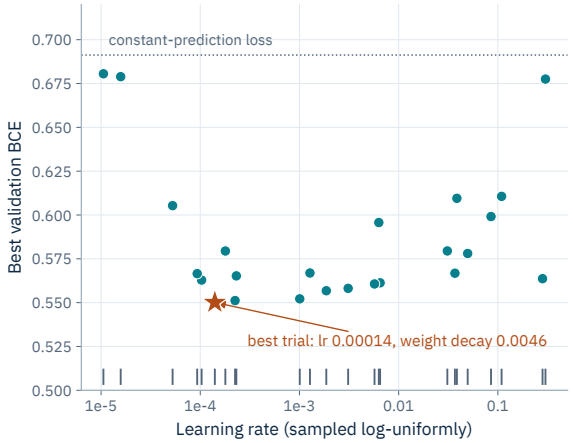
Cross-reference ML Module 5 for the search strategies themselves, including successive halving and Bayesian optimization, and for nested cross-validation. [2 min] @source https://www.deeplearningbook.org/ ; https://github.com/google-research/tuning_playbook

Sample learning rates on a log scale

OPTIMIZERS, SCHEDULES, AND TUNING

Sample learning rates on a log scale

- Draw log10 of the rate uniformly: each decade gets equal attention
- A wide first pass finds the usable window; then narrow it
- Losses within about 0.01 may be seed noise: repeat the best few
- Keep split, budget, and seed policy fixed across trials



Computed: 24 random trials of learning rate and weight decay (numpy seed 7) on the synthetic ESP table; best validation loss with patience 15.

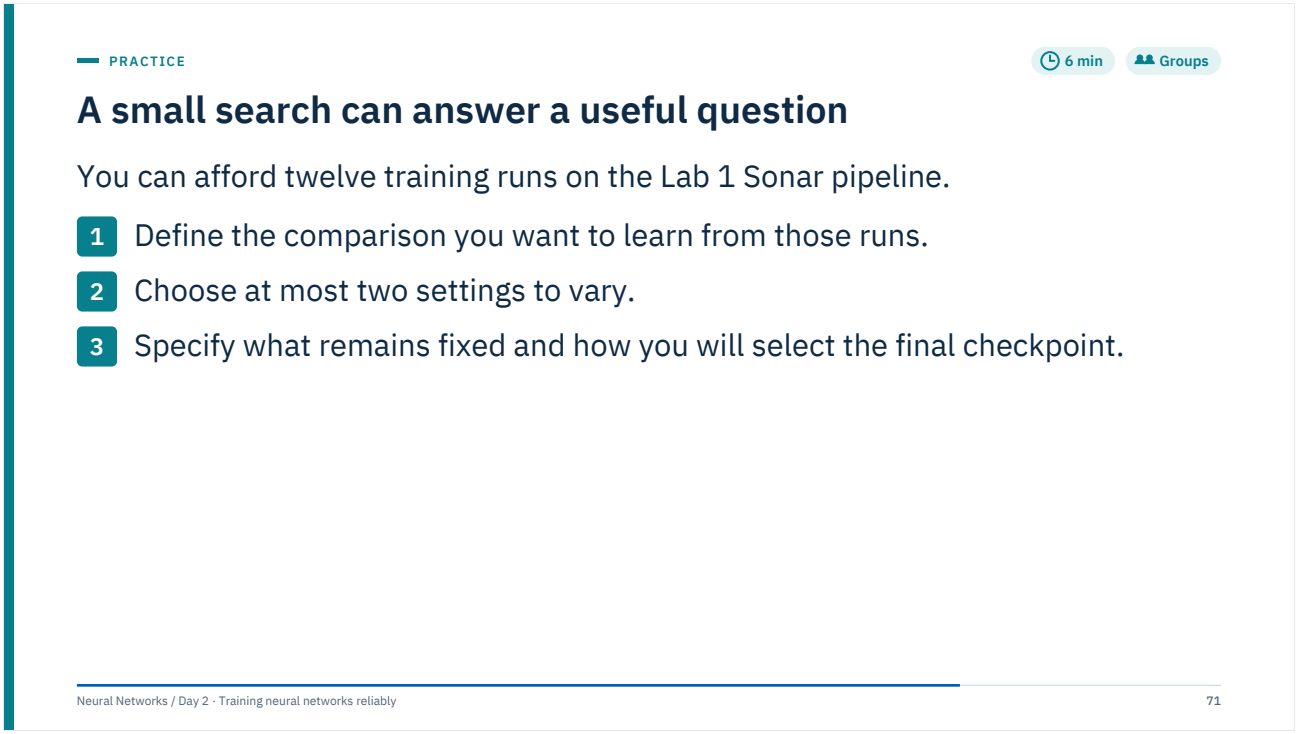
Neural Networks / Day 2 · Training neural networks reliably 70

Twenty-four trials drew the learning rate log-uniformly between 1e-5 and 0.32 and the AdamW weight decay log-uniformly between 1e-4 and 3.2. Sampling uniformly in log space puts as many trials between 1e-5 and 1e-4 as between 0.01 and 0.1; sampling uniformly in the raw range would put almost every trial above 0.03.

The result has the typical shape: rates below about 3e-5 barely learn (loss near the constant-prediction level of 0.69), rates between 1e-4 and 1e-2 mostly reach 0.55 to 0.60, and larger rates give more variable results. The best trial (learning rate 1.4e-4, weight decay 0.0046, loss 0.550) is within 0.01 of four other trials, well inside the seed-to-seed spread: rerunning the best setting with six seeds gives best losses from 0.551 to 0.576. Treat the best trial as a region to refine and confirm, not a winner.

Bergstra and Bengio (2012) explain why random search covers the important dimensions better than a grid; ML Module 5 covers this in depth. [3 min] @source https://jmlr.org/papers/v13/bergstra12a.html

A small search can answer a useful question



Groups of three, four minutes, then two groups present. [6 min]

A useful plan might vary the learning rate and weight decay with matched data splits, a fixed architecture and a few repeated seeds. Require a primary metric (validation loss or balanced accuracy on the fixed 42 rows) and a budget stated in epochs and patience. A wide collection of unrelated models is harder to interpret than a controlled experiment.

Strong answers state the question first ("Is the default learning rate of 1e-3 within the good window, and is the difference between the two best rates larger than seed noise?"), then allocate runs to answer it. The next slide shows one such plan.

Checkpoint selection: early stopping on validation loss with the copied checkpoint, as in the lab; the final test set is used once, after the choice. Weak answers vary four settings once each, or use the final rows to pick the winner.

A twelve-run plan that answers one question

DISCUSSION

A twelve-run plan that answers one question

Runs	What varies	Seeds	Decision rule
1–4	learning rate 3e-4, 1e-3, 3e-3, 1e-2	0	keep the best two
5–12	the best two rates	1–4 each	compare mean and spread
after	final assessment of the chosen setting	—	once, no changes

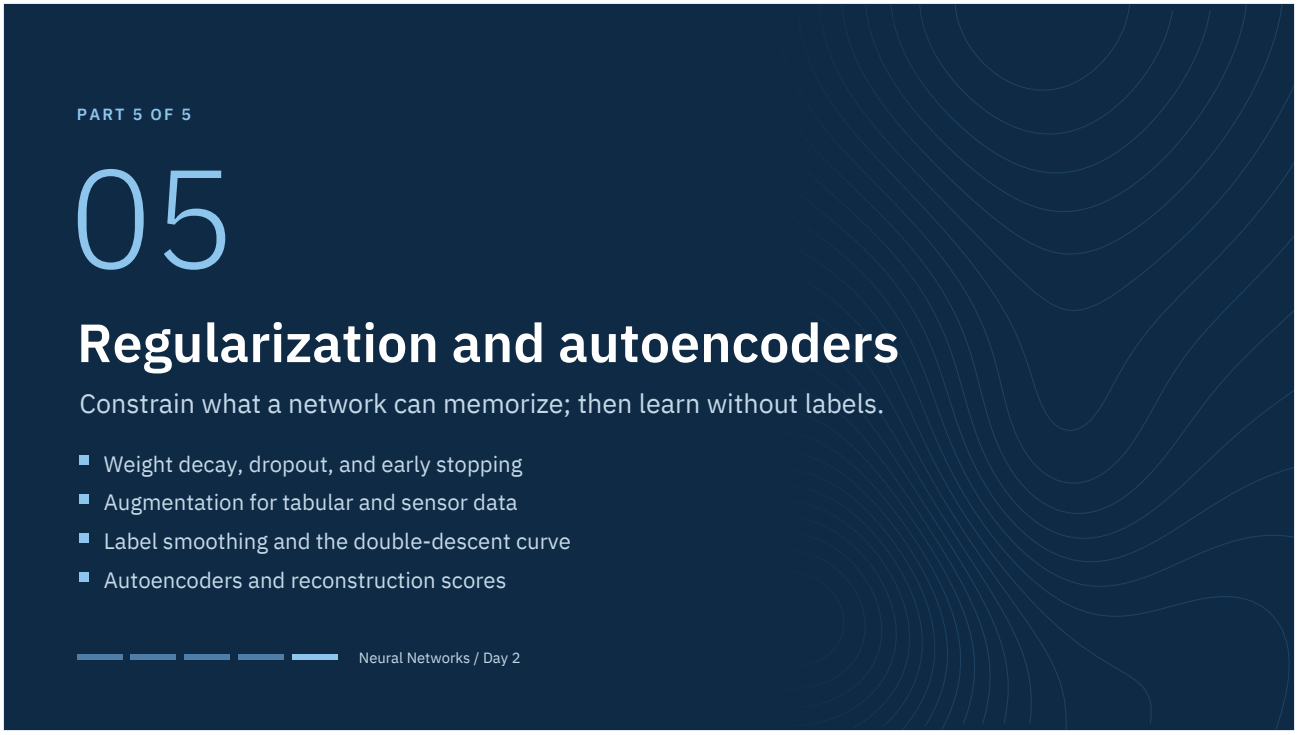
Fixed throughout: split, width 32, dropout 0.2, weight decay 1e-3, batch 16, 80 epochs, patience 10.

Four exploratory runs span a factor of about 30 in learning rate on a log grid; that is enough to find the usable window for this small model. The remaining eight runs spend the budget on repetition: four more seeds for each of the two best rates, five runs per rate including the exploratory one. With five runs each you can compare means and see whether the ranges overlap; with 42 validation rows, overlapping ranges mean you cannot claim a difference.

Everything else is fixed and written down before the first run, including the stopping rule, so the only thing the twelve runs can tell you about is the learning rate.

Alternative good plans: two rates by two weight decays by three seeds (also twelve runs), or three rates by four seeds. The final assessment on the reserved rows is not one of the twelve: it happens once, after the decision, and cannot change it. [2 min]

Regularization and autoencoders



Part 5 runs from about 14:50 to 15:35, straight into Lab 2. The common thread is constraint. Regularization techniques constrain what a network can memorize from a small training set: smaller weights, noisy activations, fewer effective training steps, more varied examples, softer targets. An autoencoder applies a different constraint, a narrow bottleneck, and uses it to learn the structure of normal data without any labels.

Both matter in energy work, where labelled events are scarce and unlabelled sensor data are plentiful. [1 min]

Regularization controls different mechanisms

REGULARIZATION AND AUTOENCODERS

Regularization controls different mechanisms

Technique	Mechanism	What to tune
Weight penalty / decay	Discourages large parameter values	Penalty strength
Dropout	Perturbs hidden activations	Drop probability
Early stopping	Chooses a checkpoint before validation worsens	Patience and monitored metric
Data augmentation	Adds valid training variations	Label-preserving transforms
Label smoothing	Softens one-hot targets	Smoothing amount ε

Neural Networks / Day 2 · Training neural networks reliably 74

Walk the table as a map for the next slides. Each technique acts on a different part of training: the parameters (weight decay), the hidden activations (dropout), the length of training (early stopping), the data (augmentation) and the targets (label smoothing). Because they act on different things they can be combined, but each adds a setting to tune, so add them one at a time and keep the ones that help on validation data.

Weight decay in AdamW is decoupled from the adaptive gradient update; it is not generally equivalent to adding an L2 penalty under Adam, as Part 4 showed. Data augmentation must be specific to what should preserve the target: a transformation that is harmless for photographs can be physically meaningless for a pressure trace. [2 min]

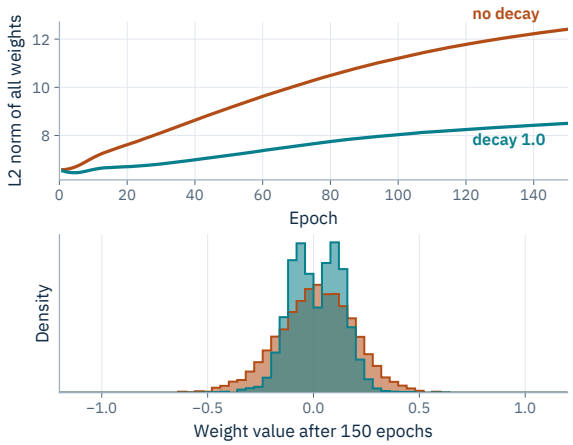
Weight decay pulls every weight toward zero

REGULARIZATION AND AUTOENCODERS

Weight decay pulls every weight toward zero

$$\tilde{L} = L + \frac{\lambda}{2}\|w\|^2 \quad \Rightarrow \quad \nabla\tilde{L} = \nabla L + \lambda w$$

- The penalty adds λ *w* to each gradient; AdamW shrinks *w* directly
- Smaller weights make smoother functions of the inputs
- Here decay 1.0 limited the weight norm to 8.5 instead of 12.4



Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW lr 1e-3 with weight_decay 0 or 1.0, 150 epochs, torch seed 0.

Neural Networks / Day 2 · Training neural networks reliably 75

The equation is the classical L2 penalty: λ sets its strength and ||w||² is the sum of squared weights. Its gradient adds λw to every weight's gradient, so each step pulls every weight a little towards zero. AdamW implements the pull directly, shrinking every weight by a factor (1 − ηλ) per step: with η = 0.001 and weight_decay = 1.0, that is 0.1% per step.

The figure trains the same network with and without decay. Without it, the norm of all weights nearly doubles, from 6.6 to 12.4, as the network memorizes; with it the norm grows only to 8.5, and the weight distribution is narrower. The validation loss after 150 epochs is 1.60 without decay and 0.90 with it; the best validation loss (0.56, at epoch 11) is the same in both, so decay here mainly makes the model less sensitive to training too long.

A weight decay of 1.0 is large; PyTorch's AdamW default is 0.01. The right value depends on the learning rate and the data size: tune it on a log scale. [3 min]

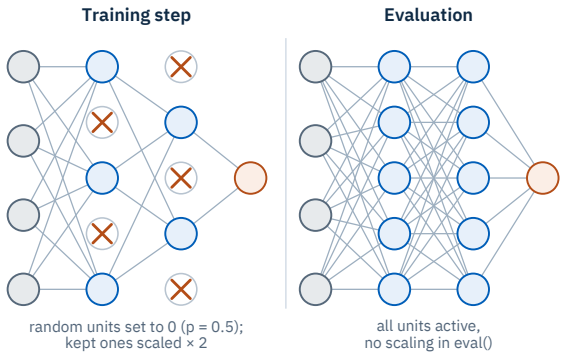
Dropout changes training-time activations

REGULARIZATION AND AUTOENCODERS

Dropout changes training-time activations

$$\tilde{a} = \frac{m \odot a}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p)$$

- During training, each activation is kept with probability 1–*p*
- Inverted dropout rescales kept activations to preserve their expectation
- During evaluation, dropout is disabled



a new random mask is drawn for every mini-batch

Schematic of inverted dropout with drop probability p = 0.5.

Neural Networks / Day 2 · Training neural networks reliably 76

Symbols: a is the vector of activations, m a random mask of zeros and ones drawn independently for each unit, p the drop probability, ⊙ element-wise multiplication. In PyTorch, nn.Dropout(p) uses p as the probability of dropping; Keras's rate means the same.

Because each unit is kept with probability 1 – p, dividing by 1 – p keeps the expected activation unchanged. With p = 0.5 the kept units are doubled during training, and at evaluation time nothing needs rescaling. A new mask is drawn for every mini-batch, so the network cannot rely on any single unit and effectively trains many overlapping sub-networks (Srivastava et al., 2014).

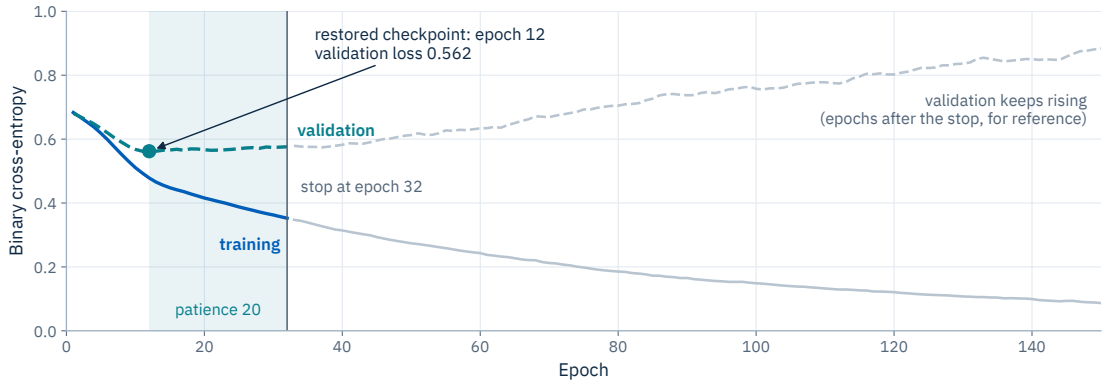
Dropout creates stochastic training noise; it does not delete learned parameters. Avoid applying it indiscriminately in every layer: for small tabular networks a rate of 0.1 to 0.3 after hidden layers is a common starting point, and it is usually unnecessary right before the output. [2 min] @source https://jmlr.org/papers/v15/srivastava14a.html

Early stopping keeps the best validation checkpoint

REGULARIZATION AND AUTOENCODERS

Early stopping keeps the best validation checkpoint

The patience rule decides when to stop; the copied checkpoint decides what you keep.



Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW (weight decay 0.1, dropout 0.2), torch seed 0; patience 20 applied to the recorded curve; grey = epochs after the stop.

This is a real training run, recorded for 150 epochs so that we can see what early stopping avoids. The validation loss reaches its minimum of 0.562 at epoch 12. With a patience of 20, training stops at epoch 32, the first epoch with 20 consecutive non-improving evaluations, and the checkpoint from epoch 12 is restored. The grey curves show what would have happened without stopping: training loss keeps falling towards 0.09 while validation loss climbs to about 0.88.

Early stopping is regularization by limiting the number of effective updates. It needs a validation set that the final report does not rely on, because choosing the epoch is itself a form of fitting: the validation loss at the chosen epoch is optimistic.

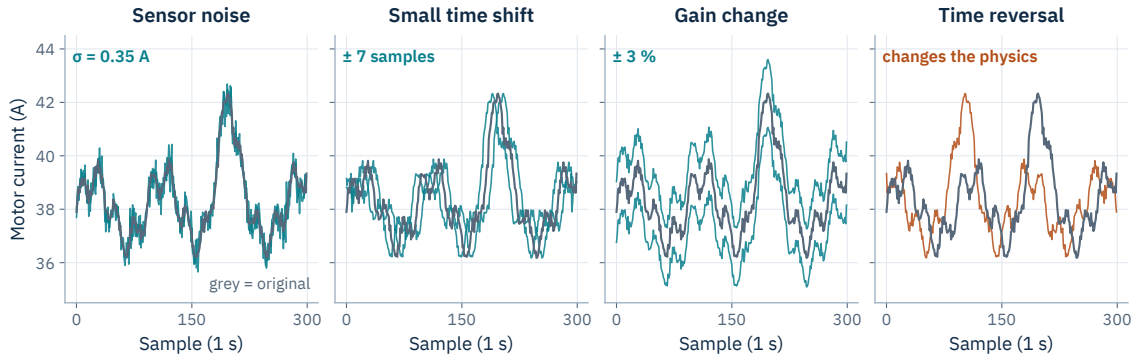
Patience is a trade-off: too short stops on noise, too long wastes compute but costs no accuracy, because the best checkpoint is kept. [2 min]

Sensor augmentation must preserve the label

REGULARIZATION AND AUTOENCODERS

Sensor augmentation must preserve the label

Add only variation the real sensors could produce under the same operating condition.



Computed: synthetic ESP motor-current trace (numpy seeds 4 and 1); noise level, shift, and gain are illustrative, not sensor specifications.

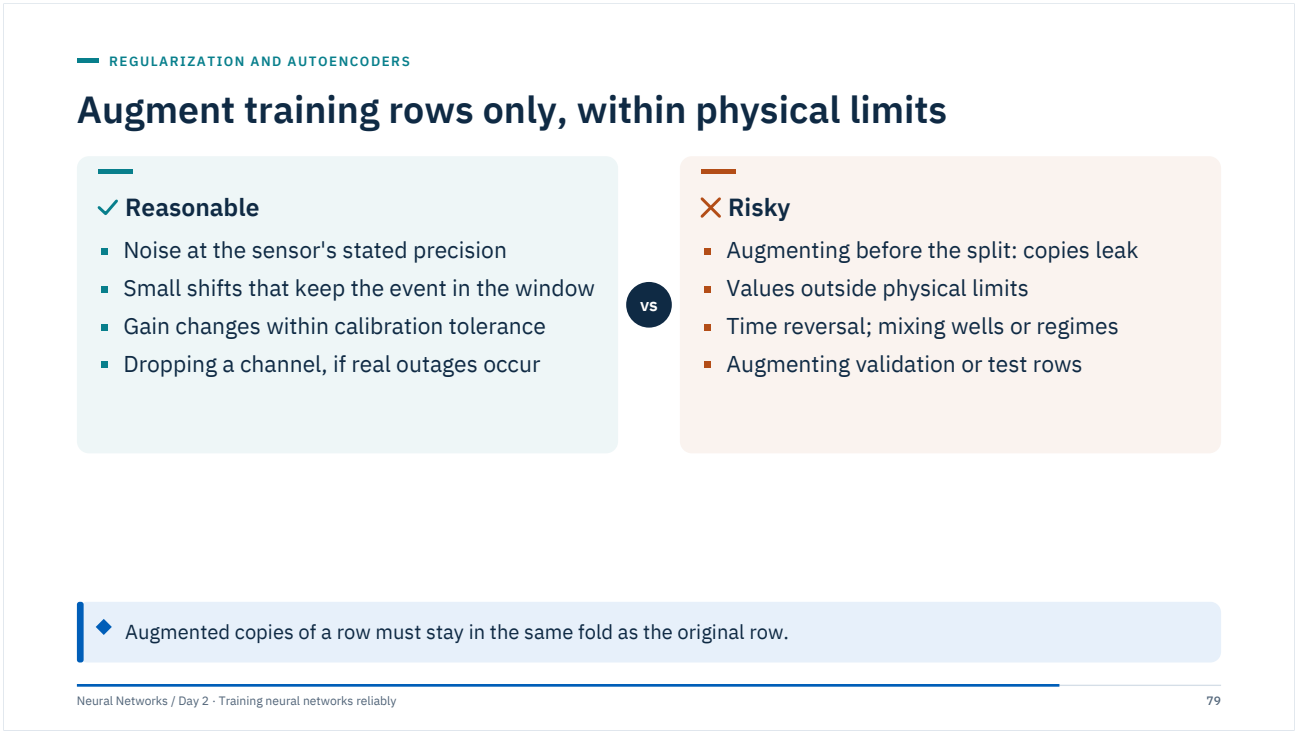
Neural Networks / Day 2 · Training neural networks reliably 78

Augmentation creates modified copies of training examples that should have the same label. For images the standard moves are flips and crops (Day 4). For tabular and sensor data the valid moves come from physics and instrumentation.

Sensor noise: add Gaussian noise at the instrument's stated precision; here 0.35 A on a current of about 38 A. Small time shift: move a window by a few samples so that the model does not depend on the event starting at an exact index. Gain change: scale by a few percent within the calibration tolerance of the sensor.

The last panel is a warning. Reversing the trace in time turns a gradual load increase followed by an exponential decay into a sharp rise followed by a slow ramp, a transient the pump would not produce. For a model that learns dynamics, that example has the wrong physics even if it has the right label. Ask learners which of their own signals have a direction in time, and which transformations their instruments could actually produce. [3 min]

Augment training rows only, within physical limits



The rules are the usual leakage rules (covered in ML Module 1) applied to augmentation. Split first, then augment only the training rows; if an augmented copy of a training row lands in validation, the validation score measures memorization. With grouped splits (by well, pump or field), every copy belongs to the group of its original.

Keep augmented values inside physical limits: negative flow rates, pressures above the pump rating or vibration below zero teach the model about situations that cannot occur. Mixing data from different wells or regimes to create new examples (as mixup does for images) can blur exactly the differences the model should learn.

Never augment validation or test data to make the numbers look bigger; evaluation must reflect real data. Test-time augmentation, averaging predictions over transformed copies at inference, is a separate technique with its own validation. [2 min]

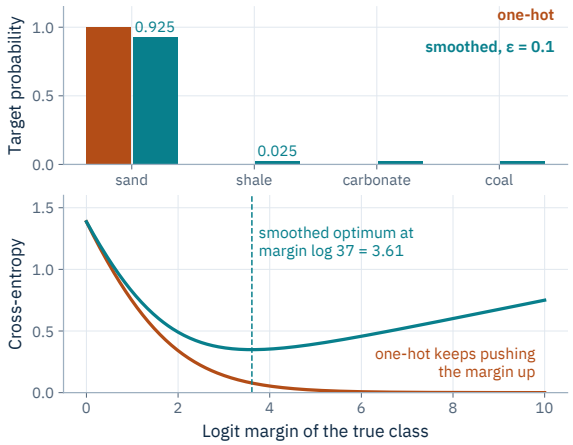
Label smoothing softens overconfident targets

REGULARIZATION AND AUTOENCODERS

Label smoothing softens overconfident targets

$$y_k^{\mathrm{LS}} = (1-\varepsilon)\, y_k + \frac{\varepsilon}{K}$$

- For K = 4 and ε = 0.1 the targets become 0.925 and 0.025
- The loss no longer rewards ever-larger logits
- PyTorch: `label_smoothing=0.1` in the loss
- Check calibration on validation data; probabilities shift



Exact targets and cross-entropy for four classes with smoothing 0.1; other logits held equal.

Neural Networks / Day 2 · Training neural networks reliably 80

Symbols: y_k is the one-hot target for class k, K the number of classes, ε the smoothing amount. With four facies classes and ε = 0.1, the true class target is 0.9 + 0.1/4 = 0.925 and each other class gets 0.025.

The bottom panel shows why this regularizes. With one-hot targets the cross-entropy keeps falling as the true class's logit margin grows, so the network is always rewarded for becoming more confident. With smoothed targets the loss has a minimum at a margin of log(0.925 / 0.025) = log 37 = 3.61 and rises beyond it, which discourages extreme logits.

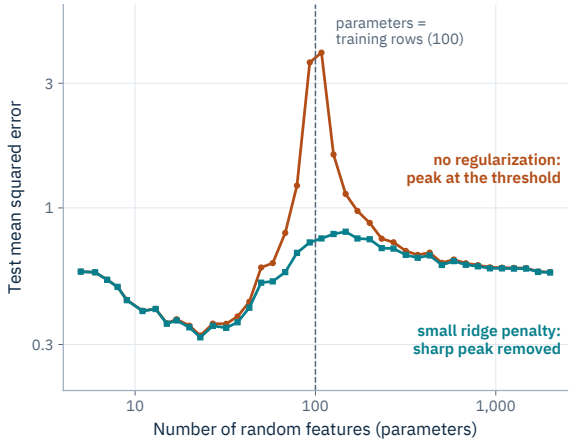
Label smoothing is a single argument in PyTorch's CrossEntropyLoss. It was popularized by Szegedy et al. (2016) for image classification. Its effect on probability calibration varies, so check reliability on validation data (calibration is covered in ML Module 5), and do not use it when you need the raw probabilities to reflect label frequencies without further calibration. [2 min] @source https://arxiv.org/abs/1512.00567 ; https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

Test error can peak where parameters equal rows

REGULARIZATION AND AUTOENCODERS

Test error can peak where parameters equal rows

- Classical U-curve: error falls, then rises with capacity
- At the threshold, the fit is forced through every noisy row
- Beyond it, error can fall again; a small penalty removes the spike
- Shown for random-feature regression; not a guarantee for your model



Computed: random ReLU features fitted by minimum-norm least squares (orange) or ridge 0.1 (teal) to 100 noisy synthetic rows; numpy seed 0.

Neural Networks / Day 2 · Training neural networks reliably 81

This is a computed demonstration of the double-descent curve (Belkin et al., 2019). A regression model with p random ReLU features is fitted to 100 noisy training rows. With few features, test error falls as capacity grows, reaching about 0.32 near 23 features. It then rises sharply to about 4 near p = 100, where the number of parameters equals the number of training rows: the model has exactly enough freedom to pass through every noisy point and must bend wildly to do so. Beyond that threshold, the minimum-norm solution spreads the fit over many parameters and test error falls again, to about 0.57 at 2,000 features in this example, though not below the first minimum.

A small ridge penalty (teal) removes the sharp spike, leaving only a mild bump (about 0.8 near 150 features). The lessons: the classical U-curve is not the whole story for over-parameterized models, and regularization and early stopping matter most near the threshold. Deep networks can show similar behaviour (Nakkiran et al., 2019), but do not assume a bigger model is automatically better for your data. [2 min] @source https://arxiv.org/abs/1812.11118 ; https://arxiv.org/abs/1912.02292

Match the regularizer to the symptom

PRACTICE

5 min Pairs

Match the regularizer to the symptom

For each case, choose a first remedy and the evidence you would check afterwards.

1. Training loss 0.05, validation loss rising after epoch 15, only 300 rows.
2. Validation accuracy is fine, but wrong predictions come with probabilities of 0.999.
3. A model trained on one field fails on sensors with a 2% gain offset.

Neural Networks / Day 2 · Training neural networks reliably 82

Three minutes in pairs, two to debrief. [5 min]

1. Classic overfitting on a small dataset. First remedy: early stopping with a copied checkpoint (restore epoch 15 or so), then weight decay or dropout, tuned on validation. More data or simpler features also help. Evidence: the gap between training and validation loss shrinks and the best validation loss is the same or better; check over several seeds.

2. Overconfidence. Label smoothing, or post-hoc calibration such as temperature scaling fitted on validation data (ML Module 5 covers calibration). Evidence: a reliability plot or expected calibration error on validation; accuracy should be roughly unchanged.

3. A distribution shift between fields. Gain augmentation within a realistic tolerance (for example ±2–3%) can make the model robust to calibration differences, but also validate by field (grouped split) to measure the problem honestly, and check whether the sensors need recalibration. Evidence: performance on a held-out field improves.

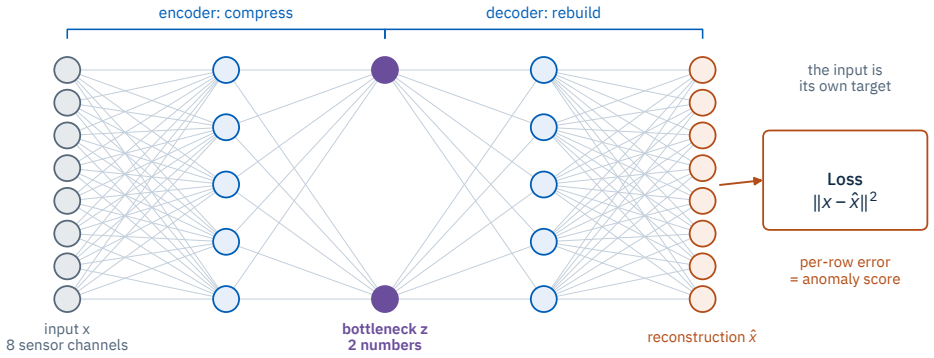
Common error: answering every case with "add dropout".

An autoencoder squeezes data through a bottleneck

REGULARIZATION AND AUTOENCODERS

An autoencoder squeezes data through a bottleneck

Trained to reproduce its own input, the network must keep only the dominant structure of the data.



Schematic of a dense autoencoder (8-5-2-5-8); the loss compares the reconstruction with the input.

Neural Networks / Day 2 · Training neural networks reliably 83

An autoencoder is two networks trained together. The encoder maps an input row, here eight sensor channels, to a small code z, here two numbers. The decoder maps z back to a reconstruction x-hat with the same shape as the input. The loss is the reconstruction error, usually the mean squared difference between x and x-hat, so no labels are needed: the input is its own target.

The bottleneck is the constraint. If eight correlated sensor channels are really driven by two operating variables, such as flow rate and pressure, two numbers are enough to reconstruct them well, and the autoencoder learns that relationship. Rows that do not follow it reconstruct badly.

Cross-references: ML Module 4 previews autoencoders as nonlinear dimensionality reduction and compares them with PCA; Day 4 builds convolutional autoencoders for images; Day 5 applies sequence autoencoders to time series. [2 min] @source https://www.deeplearningbook.org/

An autoencoder is an encoder, a decoder, and an MSE loss

REGULARIZATION AND AUTOENCODERS

An autoencoder is an encoder, a decoder, and an MSE loss

```
class AE(nn.Module):
    def __init__(self, n_in=8, k=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(n_in, 16), nn.Tanh(), nn.Linear(16, k))
        self.decoder = nn.Sequential(
            nn.Linear(k, 16), nn.Tanh(), nn.Linear(16, n_in))

    def forward(self, x):
        return self.decoder(self.encoder(x))

model = AE()
loss = nn.functional.mse_loss(model(xb), xb)    # target = input
score = ((model(x) - x) ** 2).mean(dim=1)       # per-row error
```

Train on standardized rows from normal operation only; score new rows in `eval()` mode.

Neural Networks / Day 2 · Training neural networks reliably 84

This is the architecture used for the anomaly-score figure two slides ahead, and the pattern the Lab 2 autoencoder notebook asks for (its data have two inputs and a one-number bottleneck). The encoder and decoder are ordinary Sequential stacks; the only unusual line is the loss, which compares the output with the input batch itself.

The score is the per-row mean squared error, computed with a no-grad context in practice. Standardize with statistics fitted on the normal training rows; otherwise a channel measured in kilopascals dominates the error simply because its numbers are larger.

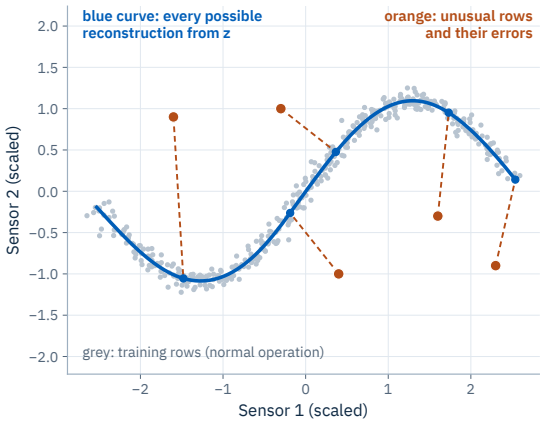
Training on normal operation only is the key assumption: the autoencoder learns what normal looks like. If the training window contains unlabelled faults, it learns to reconstruct those too. The bottleneck size k is a hyperparameter: too large and the network can copy its input, including anomalies; too small and normal rows reconstruct poorly. [2 min]

The bottleneck learns the shape of normal data

REGULARIZATION AND AUTOENCODERS

The bottleneck learns the shape of normal data

- With a one-number bottleneck, every output lies on one learned curve
- Normal rows sit near the curve and reconstruct well
- Unusual rows are pulled onto the curve: their error is large



Computed: 400 synthetic two-sensor rows near a curve (numpy seed 3); autoencoder 2-32-32-1-32-32-2 (tanh), torch seed 0; five hand-placed unusual rows.

Grey points are 400 synthetic rows from normal operation: two sensors that move together along a curved relationship, with a little noise. The autoencoder has a bottleneck of one number, so whatever it outputs must lie on a one-dimensional curve in this two-dimensional space; the blue line shows every possible reconstruction. Training has bent that curve to follow the data.

A normal row reconstructs to a nearby point on the curve, so its error is small. The five orange rows break the relationship between the sensors, even though each sensor value on its own is within the normal range. The autoencoder maps them to the curve anyway, and the dashed lines show large reconstruction errors.

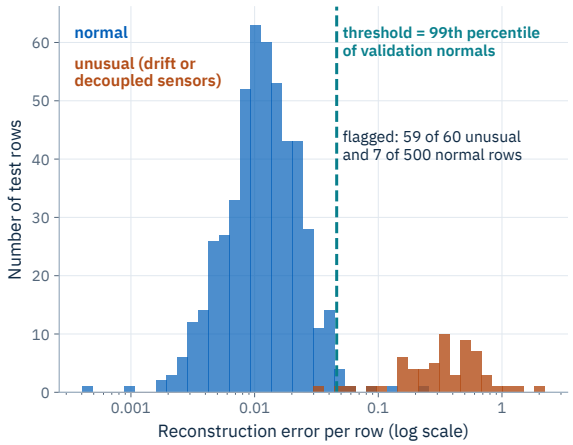
This is the geometric idea behind reconstruction-based anomaly detection, and also its limit: an anomaly that happens to lie close to the learned curve reconstructs well and is missed. [2 min]

Reconstruction error becomes an anomaly score

REGULARIZATION AND AUTOENCODERS

Reconstruction error becomes an anomaly score

- Train on normal operation; score each row by its mean squared error
- Set the threshold on validation normals, for example their 99th percentile
- Here 59 of 60 unusual rows and 7 of 500 normal rows were flagged
- Evaluate events with precision-recall (ML Module 5)



Computed: synthetic 8-channel sensor rows driven by two hidden factors (numpy seed 11); autoencoder 8-16-2-16-8 trained on 2,000 normal rows, torch seed 0.

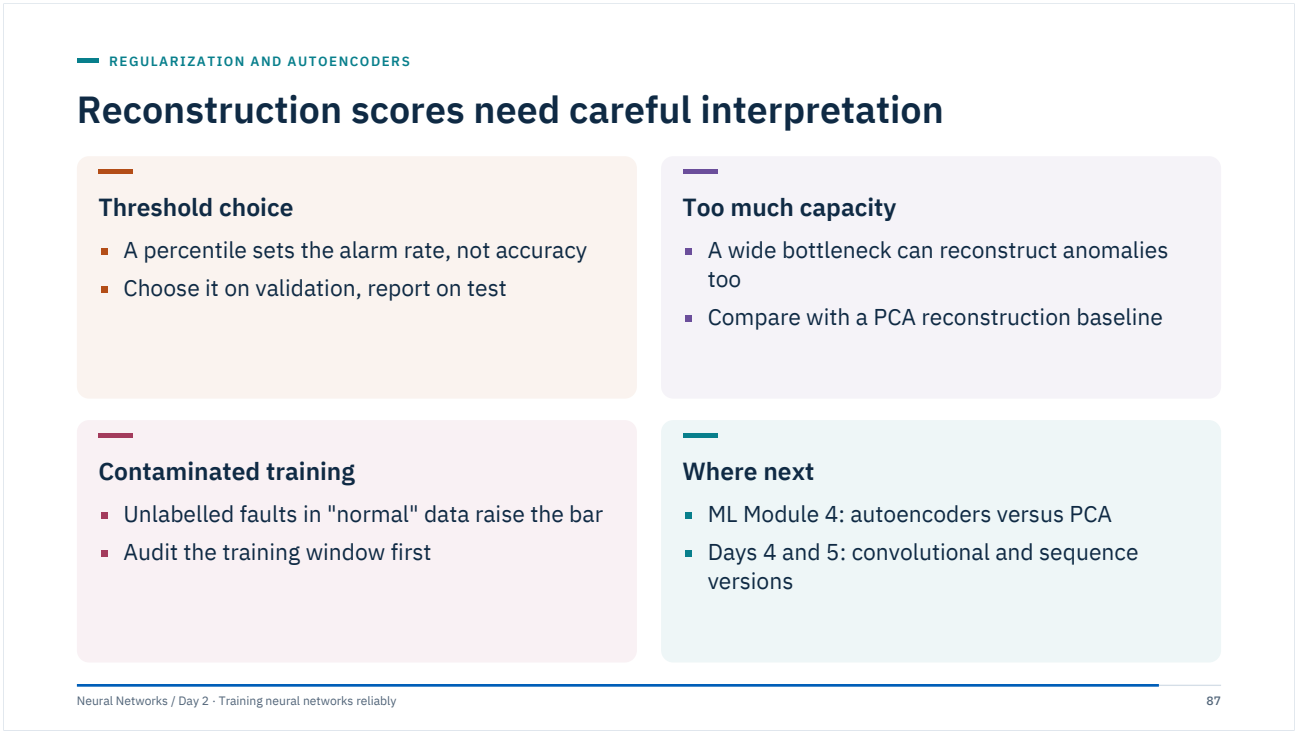
Neural Networks / Day 2 · Training neural networks reliably 86

The data are synthetic: eight sensor channels generated from two hidden operating factors plus noise. The autoencoder was trained on 2,000 normal rows only. The test set has 500 new normal rows and 60 unusual ones: 30 where one sensor drifts upward and 30 where half the channels come from a different operating state.

The threshold is the 99th percentile of reconstruction errors on 500 separate validation rows from normal operation, about 0.046. On the test set it flags 59 of the 60 unusual rows and 7 of the 500 normal ones (1.4%, close to the intended 1%).

Do not read this as field performance: the anomalies were constructed to break the learned relationship. Real faults can be subtle, rare and drifting, and the threshold is a business decision about alarm rates. Evaluate as in ML Module 5, with precision-recall curves and event-level metrics rather than accuracy. [2 min]

Reconstruction scores need careful interpretation



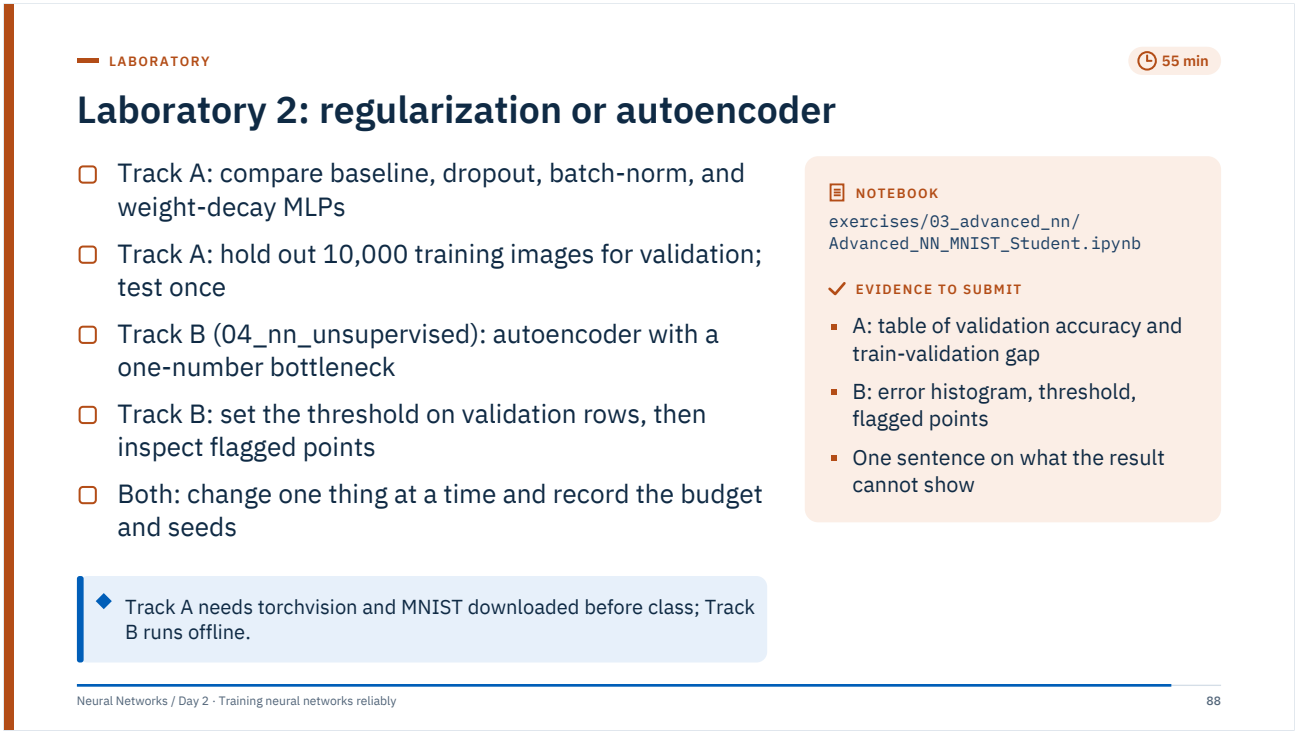
Four cautions before the lab. The threshold: choosing the 95th percentile guarantees that about 5% of normal-like rows are flagged; it is an alarm-rate decision, and it must be fixed on validation data and then reported on separate test data. The Lab 2 notebook fixes its 95th percentile on validation rows held out from the training file and applies it once to the test file.

Capacity: an autoencoder with a wide bottleneck or too many layers can learn to copy its input, including anomalies. Always compare with a simple baseline such as PCA reconstruction error (ML Modules 4 and 5); if the network does not beat it, prefer the simpler model.

Contamination: if the training window contains unrecognized faults, the autoencoder learns to reconstruct them and the scores for similar future faults stay low.

Where next: ML Module 4 compares autoencoders with PCA; Day 4 introduces convolutional autoencoders; Day 5 applies reconstruction scores to sequences. [2 min]

Laboratory 2: regularization or autoencoder



Timing: 55 minutes, then an 8-minute debrief. Learners choose one track. [55 min]

Preparation: Track A (exercises/03_advanced_nn) needs torchvision and the MNIST files (about 11 MB compressed) downloaded before class, because classroom networks are unreliable. Run the loading cell once, or point the COURSE_DATA environment variable at a folder that already holds MNIST (the default is ./data); once the files are present, the notebook does not use the network. It trains four models for 10 epochs each on 50,000 images, about 3 minutes on a recent laptop CPU and up to about 10 minutes on an older one; set NUM_EPOCHS = 5 if time is short. Track B (exercises/04_nn_unsupervised) uses the CSV files in its folder and runs offline in under a minute.

Track A protocol: the notebook holds out 10,000 training images for validation (random_split with a seeded generator), monitors them every epoch, restores each model's lowest-validation-loss checkpoint, compares the four models on validation accuracy, and evaluates the chosen one on the test set once at the end. The training accuracy used for the gap is re-measured in eval mode. Its "L2" model uses Adam's weight_decay, which is the coupled L2 form; as an extension, compare AdamW. Expected pattern: all four models reach high accuracy and differ by small margins, so report the train-validation gap and at least two seeds before ranking them.

Track B protocol: the data are uniform points in a square (labels mark a circle of radius 3), so a one-number bottleneck must fold the square onto a curve and large errors concentrate far from it. The notebook holds out 2,000 rows of the training file for validation, fits the scaler on the other 8,000, sets the threshold at the 95th percentile of the validation errors (so about 5% of normal-like rows are flagged by construction), and applies it once to the test file. Labels are not used for training; afterwards, check where flagged points lie.

Lab 2 debrief: evidence over impressions

DISCUSSION

Lab 2 debrief: evidence over impressions

Track A

- Was validation separate from test?
- Is the gap larger than seed-to-seed noise?

Track B

- Was the threshold fitted on validation rows?
- Where do the flagged points lie, and why?

Both tracks

- Budget and seeds reported
- Claims limited to this data

◆ Every claim carries its budget, its seeds, and its limits.

Neural Networks / Day 2 · Training neural networks reliably 89

Eight minutes. Ask one learner per track to present their key table or figure, then use the cards as questions. [8 min]

Track A: the four models usually land close together on MNIST; the interesting quantity is the gap between training and validation accuracy, which dropout usually narrows, and training speed, where batch normalization often helps. Reference run (solution notebook, seed 42, 10 epochs, CPU): validation accuracy 0.973 for the baseline, 0.979 with dropout, 0.981 with batch normalization and 0.975 with weight decay; train-validation gaps of 1.4 (dropout) to 1.9 points, so weight decay at 1e-4 changed little here. Batch normalization is chosen and scores 0.983 on its single test evaluation. If a learner reports a winner from one seed and differences of 0.1 points, ask for the second seed.

Track B: flagged points should concentrate where the learned one-dimensional curve cannot reach, typically corners and regions between folds of the curve. That is a property of the bottleneck and the data geometry, not of any physical anomaly; the labels (inside or outside a circle) are unrelated to the reconstruction task, which is a useful discussion point. Reference run: threshold 0.305 (95th percentile of the 2,000 validation errors, in standardized units), which flags 51 of the 1,000 test rows (5.1%), close to the intended alarm rate.

Close with the shared card: every claim should carry its budget, its seeds and its limits.

Key takeaways

KEY TAKEAWAYS

Key takeaways

- ✓ Diagnose in a fixed order: data, initial loss, one batch, gradients, validation.
- ✓ Initialization, normalization, and skip paths keep signals at a trainable scale.
- ✓ `eval()` and no-grad do different jobs; copy the best checkpoint and verify a reload.
- ✓ Learning rate first: range test, log-scale search, and a schedule on a declared clock.
- ✓ Regularize for the symptom you see, and validate every change over several seeds.
- ✓ Reconstruction error is a score; its threshold is a decision made on validation data.

Neural Networks / Day 2 · Training neural networks reliably 90

Read the six statements and ask learners to mark the one they will apply first at work. Each maps to a part of the day: the first to Part 1, the second to Part 2, the third to Part 3 and Lab 1, the fourth to Part 4, and the last two to Part 5 and Lab 2.

If time allows, ask one learner to explain the first takeaway using the motivating case: the left-hand run fails the initial-loss check; the middle run passes the first checks and fails validation; the right-hand run is what you get after the sequence, and even then it needs a test set and several seeds before anyone relies on it.

Tomorrow (Day 3) applies all of this to the capstone projects on the real course data: ESP gas lock, a photoelectric log curve and an anonymous sensor set. [3 min]

Exit check

PRACTICE

10 min · Individual

Exit check

1. What is the difference between `model.eval()` and `torch.no_grad()`?
2. Why does early stopping use validation rather than test data?
3. Why can adding L2 to Adam differ from AdamW weight decay?
4. Your initial loss on a balanced 5-class problem is 9.8. What do you check first?
5. Name one augmentation that would be invalid for your own sensor data, and why.

Neural Networks / Day 2 · Training neural networks reliably 91

Written, individual, ten minutes; collect answers or ask learners to keep them for Day 3. [10 min]

1. eval() changes the behaviour of layers such as dropout (off) and batch normalization (running statistics); no_grad (or inference_mode) stops autograd from recording operations. Evaluation needs both; neither implies the other.

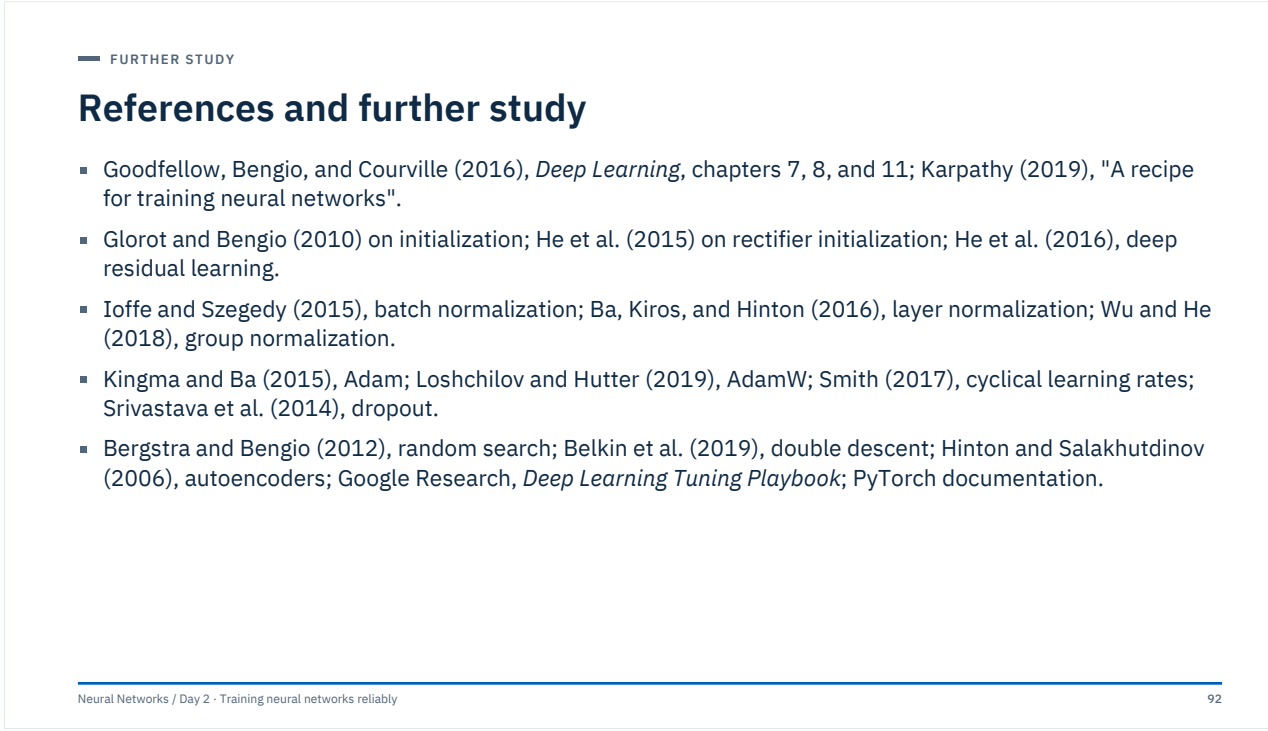
2. Choosing the stopping epoch is a model-selection decision fitted to the data used to make it. If the test set chose the epoch, the test score would be optimistic and no unbiased estimate would remain.

3. With L2 inside Adam, the penalty gradient is divided by the adaptive denominator, so weights with large gradients are barely decayed. AdamW shrinks every weight by the same fraction $\eta\lambda$ per step, separately from the adaptive update. Values are therefore not interchangeable.

4. $\log 5 = 1.609$, so 9.8 means very confident wrong predictions before training. Check input scaling first (a column in raw units), then the initialization and output layer, then the loss contract and label encoding.

5. Accept any physically argued answer: time reversal of a transient, flipping the sign of a pressure, noise far above sensor precision, mixing wells or regimes, or scaling beyond calibration tolerance.

References and further study



All references are real and verifiable; links to the papers appear in the notes of the slides where each idea is introduced. For self-study, the most useful single chapter is Goodfellow, Bengio and Courville's chapter 8 on optimization for training deep models, followed by chapter 7 on regularization. Karpathy's blog post is a practical, experience-based version of today's diagnostic sequence.

For framework practice, the PyTorch documentation pages on autograd mechanics, torch.optim (including the learning-rate schedulers), reproducibility and saving and loading models are the authoritative sources; the Keras early-stopping callback page documents restore_best_weights.

Suggested follow-up for learners who want more: the Deep Learning Tuning Playbook (Google Research, on GitHub) for a structured approach to hyperparameter tuning, and ML Module 5 for search strategies and statistical comparison of models. [1 min]

Reference: https://www.deeplearningbook.org/ ; http://karpathy.github.io/2019/04/25/recipe/ ; https://docs.pytorch.org/docs/stable/notes/autograd.html ; https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html ; https://keras.io/api/callbacks/early_stopping/